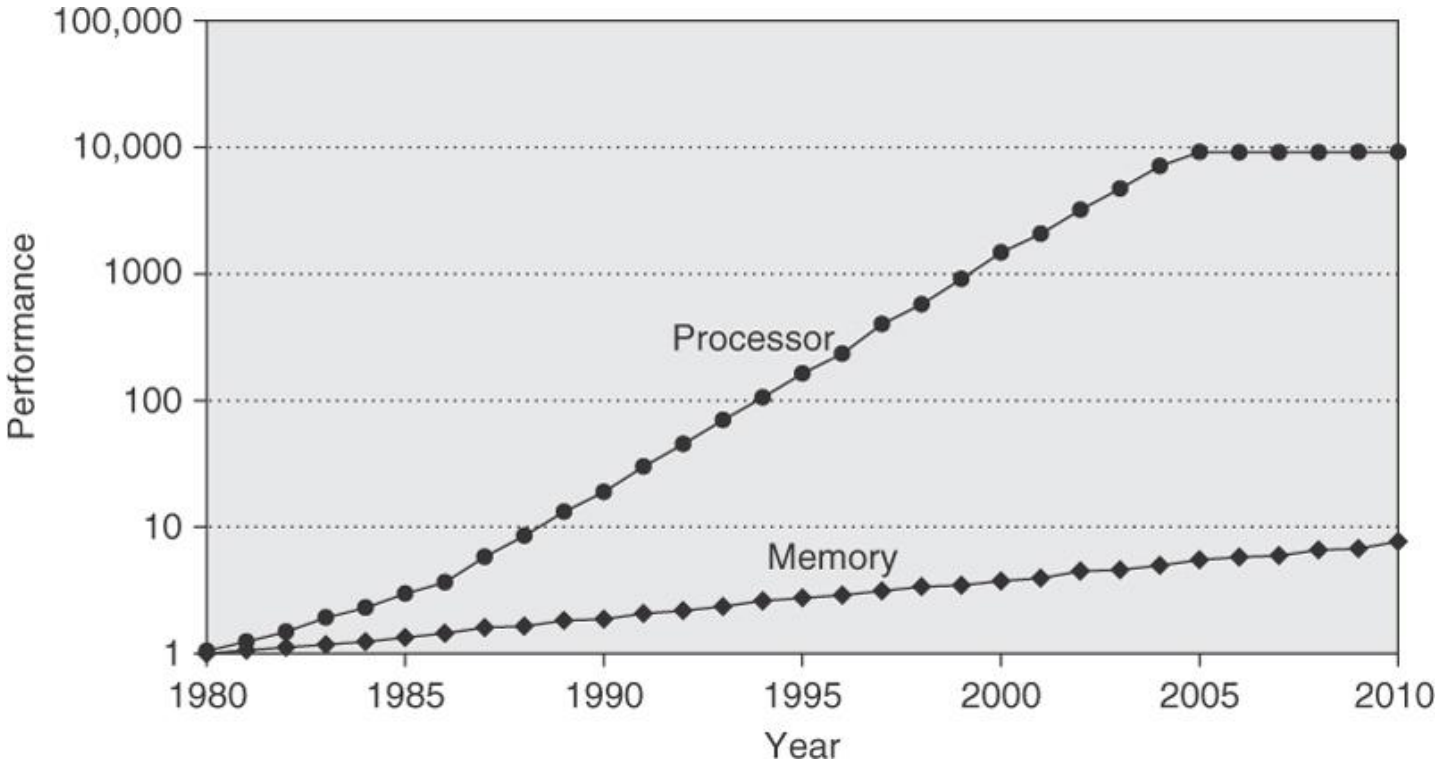
Cache memories

E. Sanchez

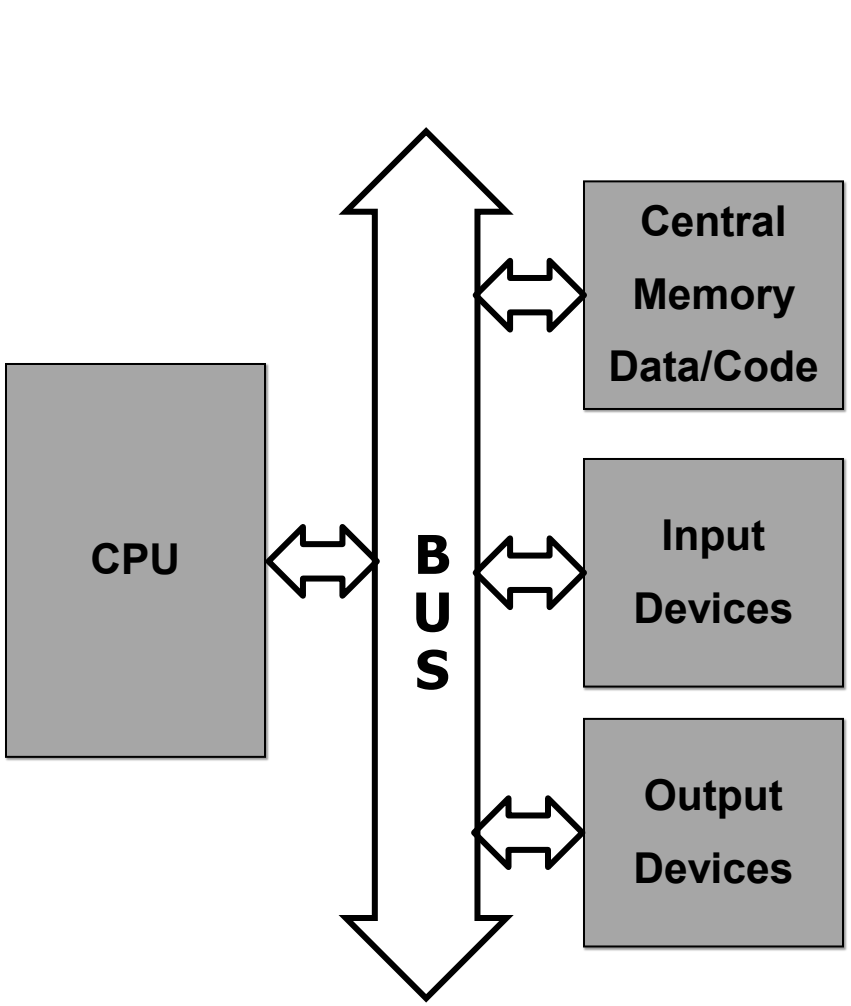
Politecnico di Torino
Dipartimento di Automatica e Informatica

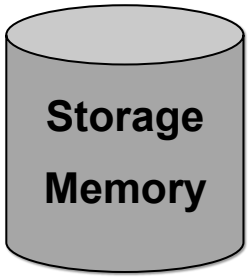
Introduction



Cache memories are small but high-speed memories that are interposed between the processor and the main memory.

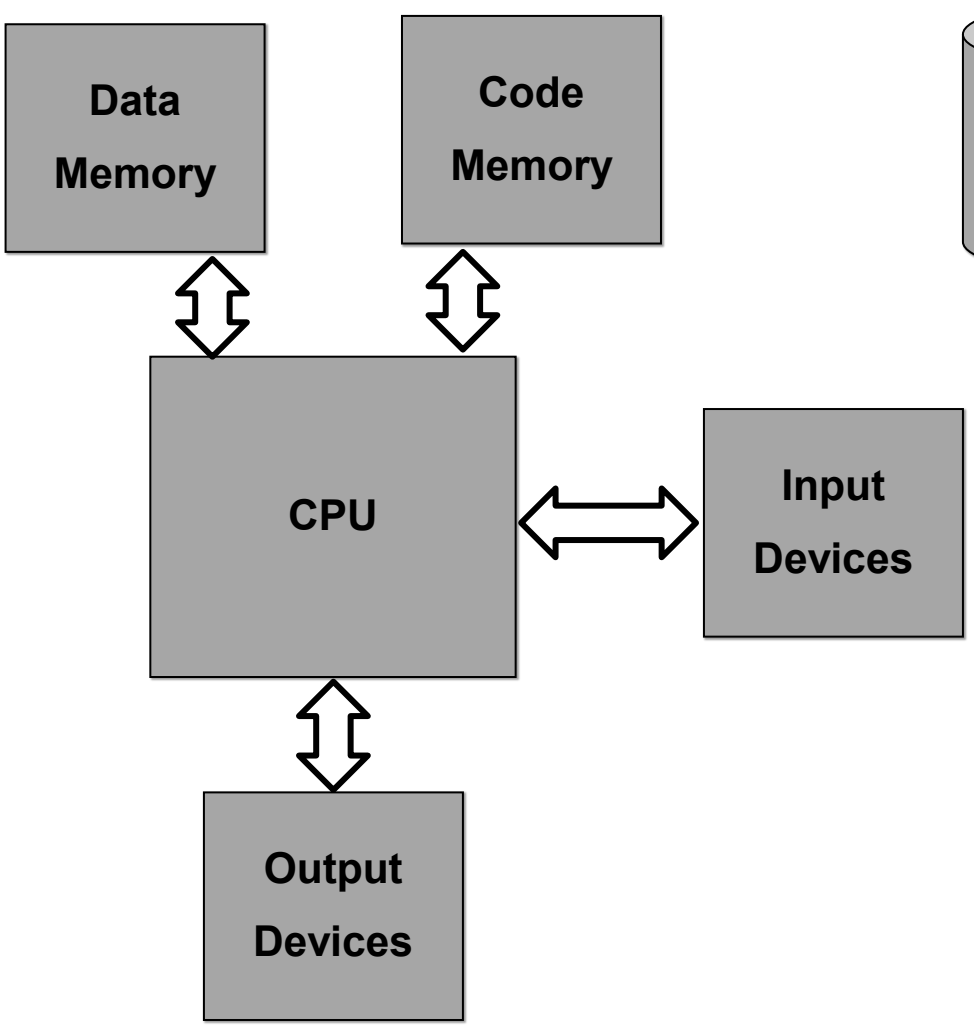
Von Neumann architecture





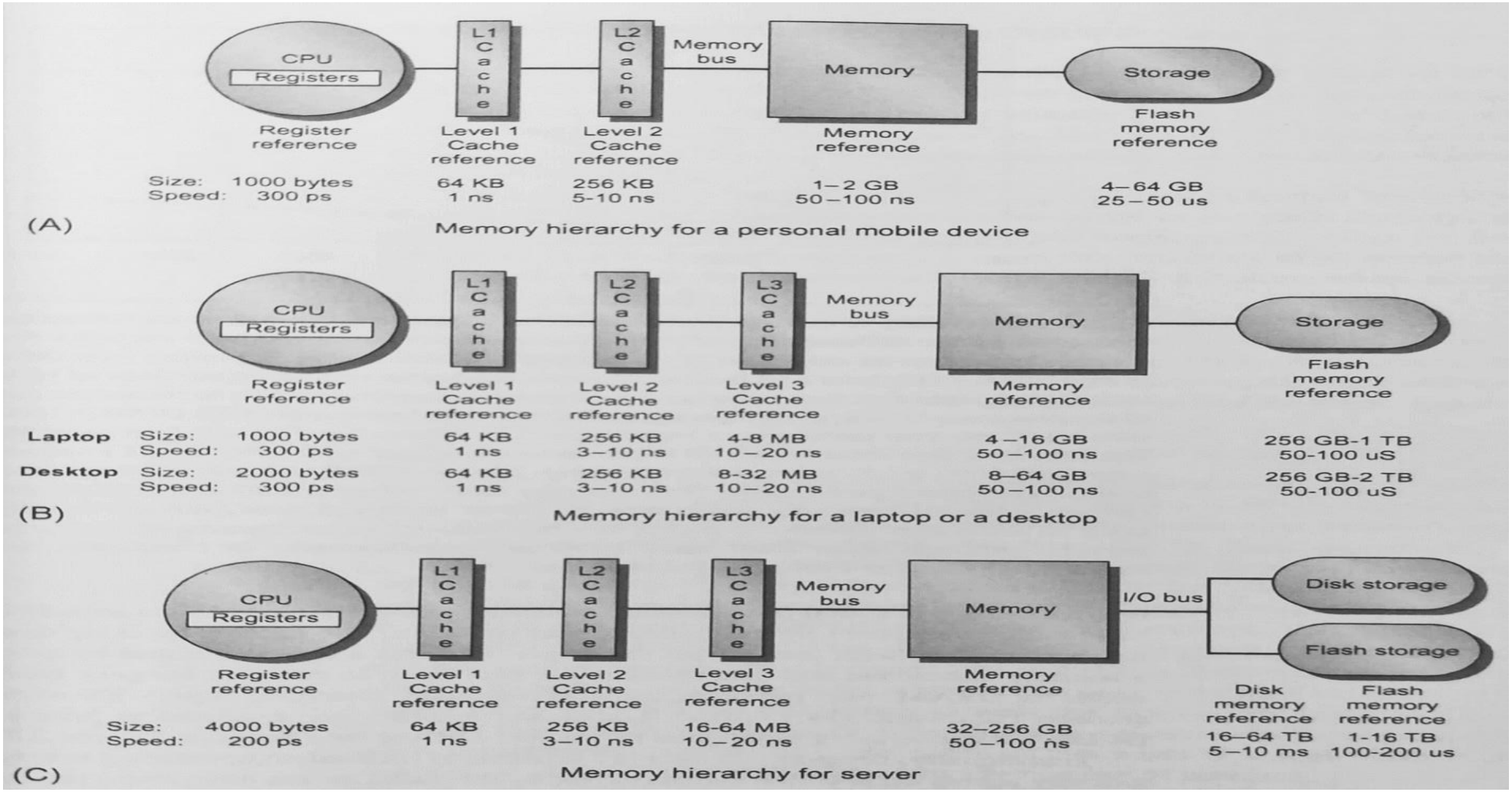
This design stores both data and instructions in the same memory, simplifying hardware design and enabling general-purpose computing.

Harvard architecture



There are two separated memories for Data and Code, these are usually cache memories.

Memory levels in a typical system



Locality of reference

The presence of a cache can improve the performance of a system due to the locality of references observed in most programs.

Locality takes two forms:

- *Temporal locality*: if at time t the program accesses to a given memory cell, it is highly probable that the program accesses again the same cell by the time $t_0 + \Delta t$
- *Spatial locality*: if at time t_0 the program accesses a memory cell with address x, it is highly likely that by the time $t_0 + \Delta t$ the program will also access the memory cell with address $x \pm e$.

Working principle

If the entire block is loaded in the cache at time t_0 (first access to a memory block), it is likely that for a certain time Δt the program will find in the cache all of the words it needs.

Performance

Let define the following elements:

- *h*: cache hit ratio
- *C*: cache access time
- *M*: memory access time when the data is not in the cache.

The average access time to memory is

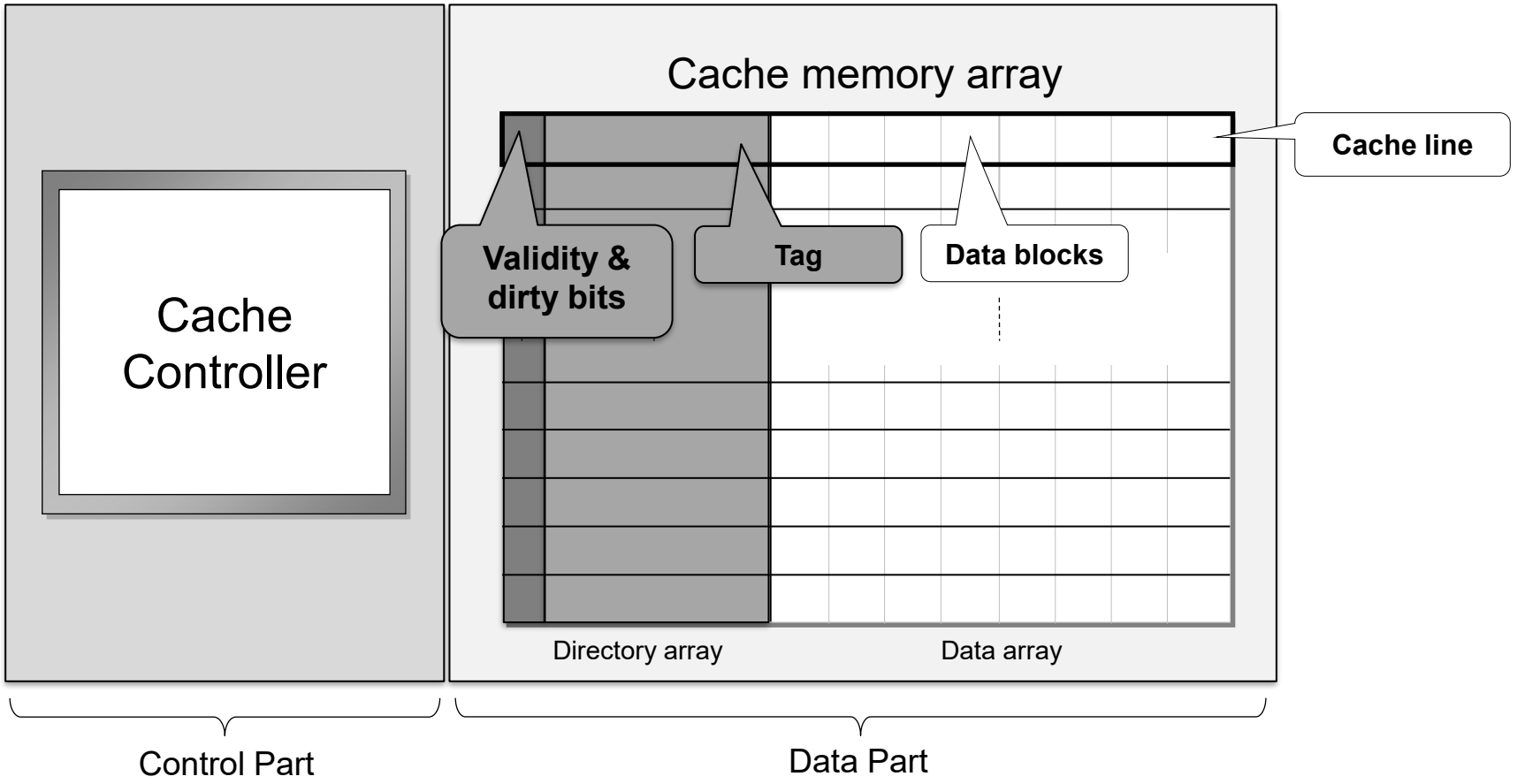
$$t_{ave} = h*C + (1-h)*M$$

Normal values for *h* are in the order of 0.8 - 0.9.

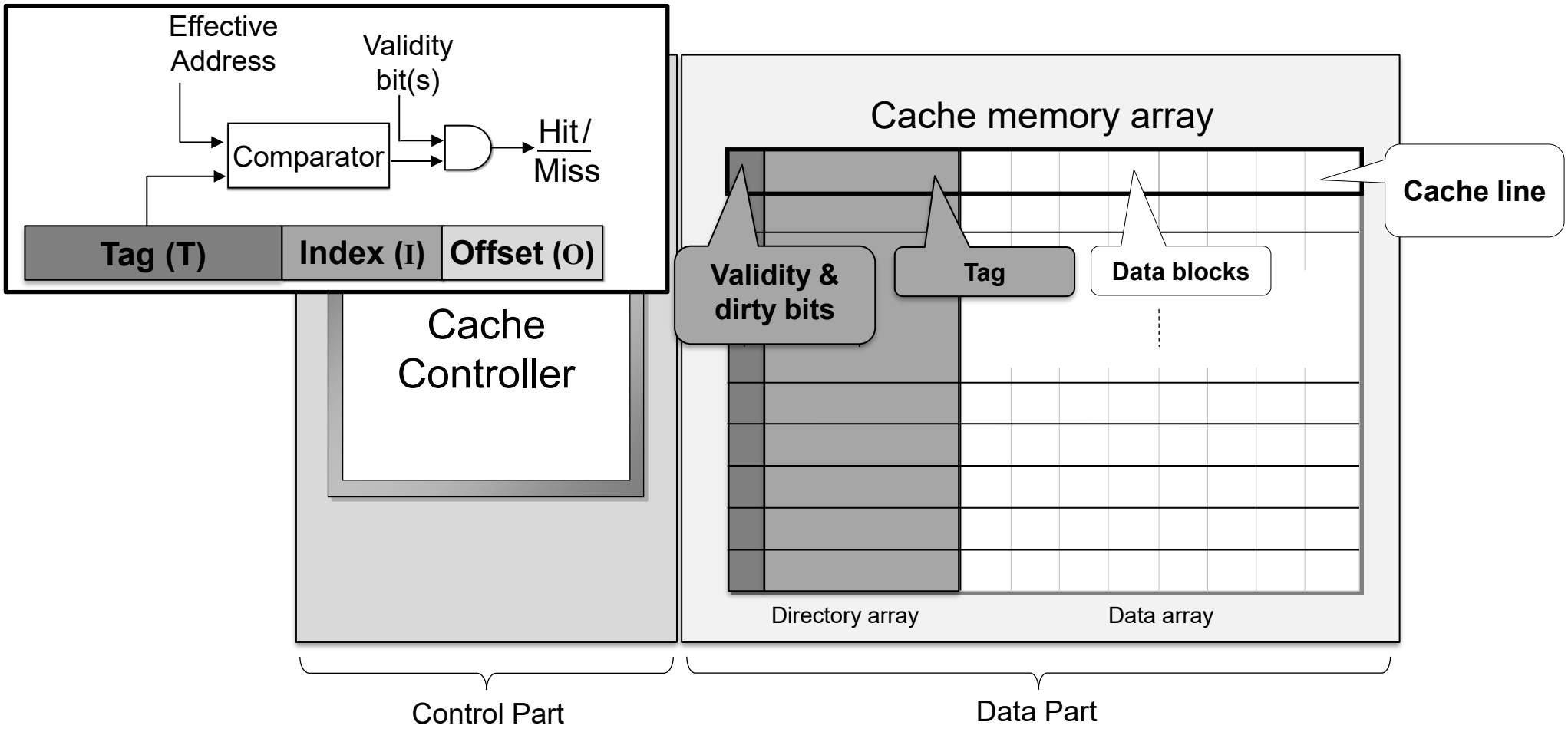
For example:

$$T_{avg}=0,8 \times 2x10^{-9} + 0,2 \times 50x10^{-6} = 10,001\ \mu s$$

Cache organization



Cache organization



Cache organization (II)

A cache is organized in lines that contain:

- A data part consisting of a one or more memory blocks where the cache information (instructions or Data) is saved;
- a tag to univocally identify the data elements;
- It may also contain some control bits indicating the status of the data elements, if the data part is valid or not, if these were modified or need to be updated.

In a multiprocessor system, the status bits may also include some coherence related bits guaranteeing data coherence and consistency.

Validity bit (V)

To achieve cache coherence in single processor systems, a validity bit is introduced for every cache line.

If it is disabled, it means that any access to the block must produce a miss.

At the power-on, the validity bits of all cache lines are disabled.

Once the data in the line is valid, the validity bit goes to One.

Dirty bit (D)

The *dirty bit D is* used to remember whether, one block of the line has been changed since it was loaded into the cache.

V and D bits operation

For example, assume a cache having a validity bit *V*, and a Dirty bit *D*.

V = 1 means the line has valid data

D = 1 means that the cache bytes are newer than the ones in the main memory

When allocating the line:

- Set V = 1, D = 0, fill in Tag and Data

When writing the line:

- Set V = 1, D = 1

When evicting one line:

- If D = 0: just set V = 0.
- If D = 1: update the Data in the main memory, then set D = 0, V = 0.

Cache organization (III)

The cache also contains the logic for

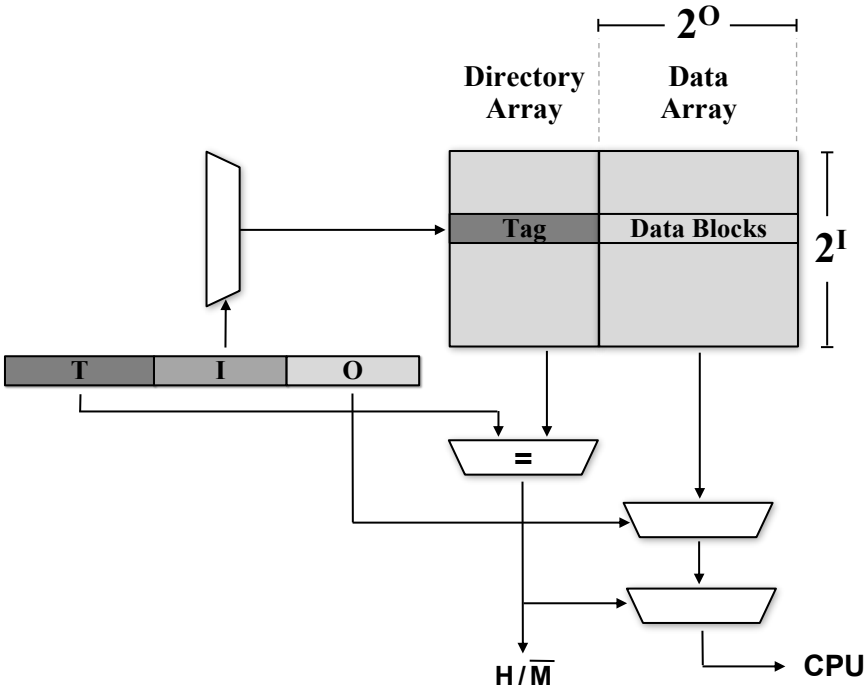
- intercepting the addresses produced by the processor
- checking inside the cache the possible presence of the block that the processor wants to access
- possibly loading the block
- When needed, replacing a cache block with a newer one.

Cache eviction refers to the process of removing data from a cache to make room for new data when the cache is full or when certain rules require old data to be discarded.

Finding a data block in the cache



Cache hit	if a required block is placed in the cache
Cache miss	if a required block is not placed in the cache



Cache behavior

The cache is located between the processor and the main memory.

Each time the processor performs an access to memory the cache intercepts the address and checks if the block to which the word belongs is in the cache, checking the value of the tags

- **if yes: it extracts the word from the block and provides it to the CPU without any access to the main memory (*hit*)**
- **if no: it loads in the cache the entire block that the word is part of (*miss*).**

Performance

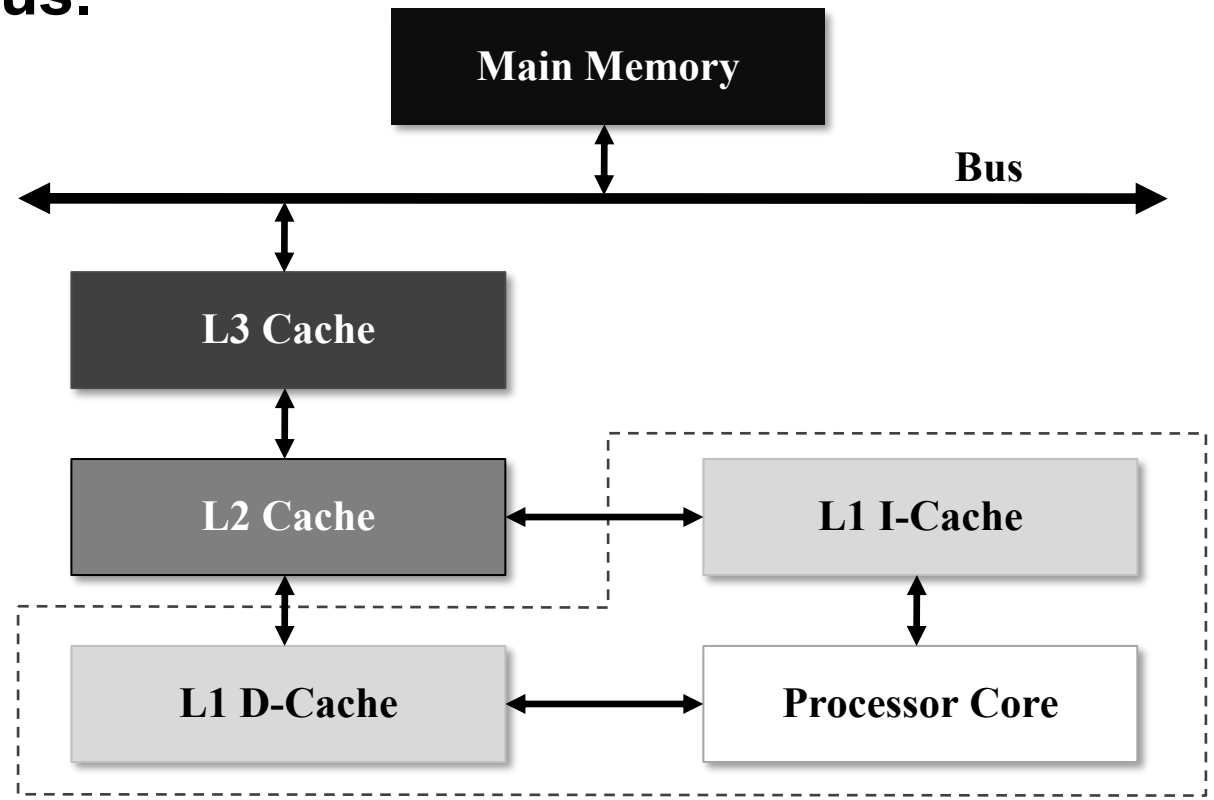
In the case of a hit, the cache reduces the memory access time by a factor that is dependent on the ratio between cache and primary memory access times.

In case of a miss, the cache may respond in two possible ways:

- It accesses the memory and loads the entire missing block; then it provides the requested word. The access time is therefore higher than in a cache-free system.
- It accesses the memory and immediately provides the requested word (*load-through* or *early restart*). This technique requires a higher cost in terms of cache hardware, but miss situations have a more limited impact on cache performance.

Cache position

The cache is normally located between the CPU and the bus, rather than between the main memory and the bus.



Cache position

The cache is normally located between the CPU and the bus, rather than between the main memory and the bus.

The benefits that are obtained in this way are:

- the bus load is reduced
- the solution is compatible with a multiprocessor architecture.

Instruction Cache and Data Cache

In some cases, there are separate caches for data and instructions; in other cases, there is only one for instructions and data.

The cache for instructions is typically easier to handle than data, as the instructions can not be changed (no write operations on the instruction cache).

Characteristic parameters

They are:

- Cache size
- Block size
- Mapping
- Replacing algorithm
- Main memory update mechanism
- Cache coherence.

Cache size

Choosing the optimal size of the cache is very important for the system cost and the performance.

As the size increases

- The cost increases
- The system performance improves
- The cache itself becomes slower.

Frequent sizes range from a few kB to a few MBs.

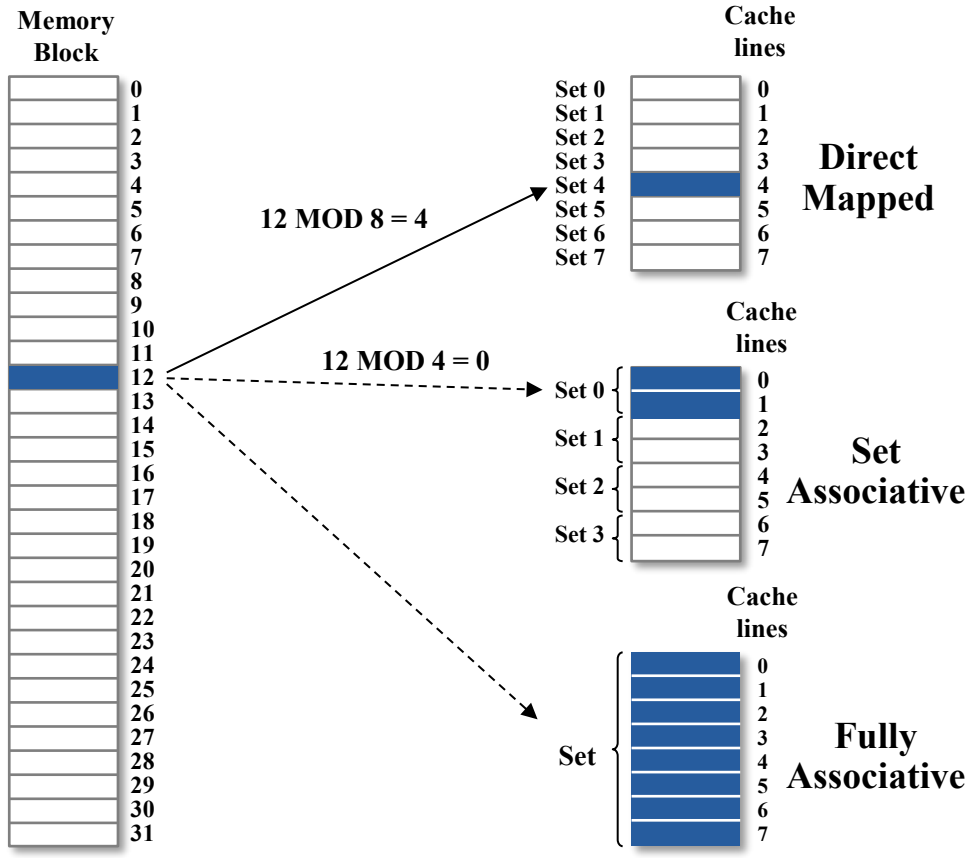
Mapping

The mapping mechanism defines in which line of the cache a certain memory block is written when it is uploaded in the cache.

You must ensure (at an acceptable cost) that the cache can quickly verify if it contains the data corresponding to a certain address.

Mapping (II)

Associativity Models



Finding the Set in the cache

of Memory Block
MOD
Total of Blocks in Cache

of Memory Block
MOD
Total of Blocks in Cache
Associativity level of cache

In a direct-mapped cache, each Cache Set corresponds to exactly one Cache Line.
If the cache associativity increases, each Cache Set contains more than one Cache Line.

Direct Mapping

Each memory block *i* is statically associated to a set *k* in the cache using the expression

$$k = i \bmod N$$

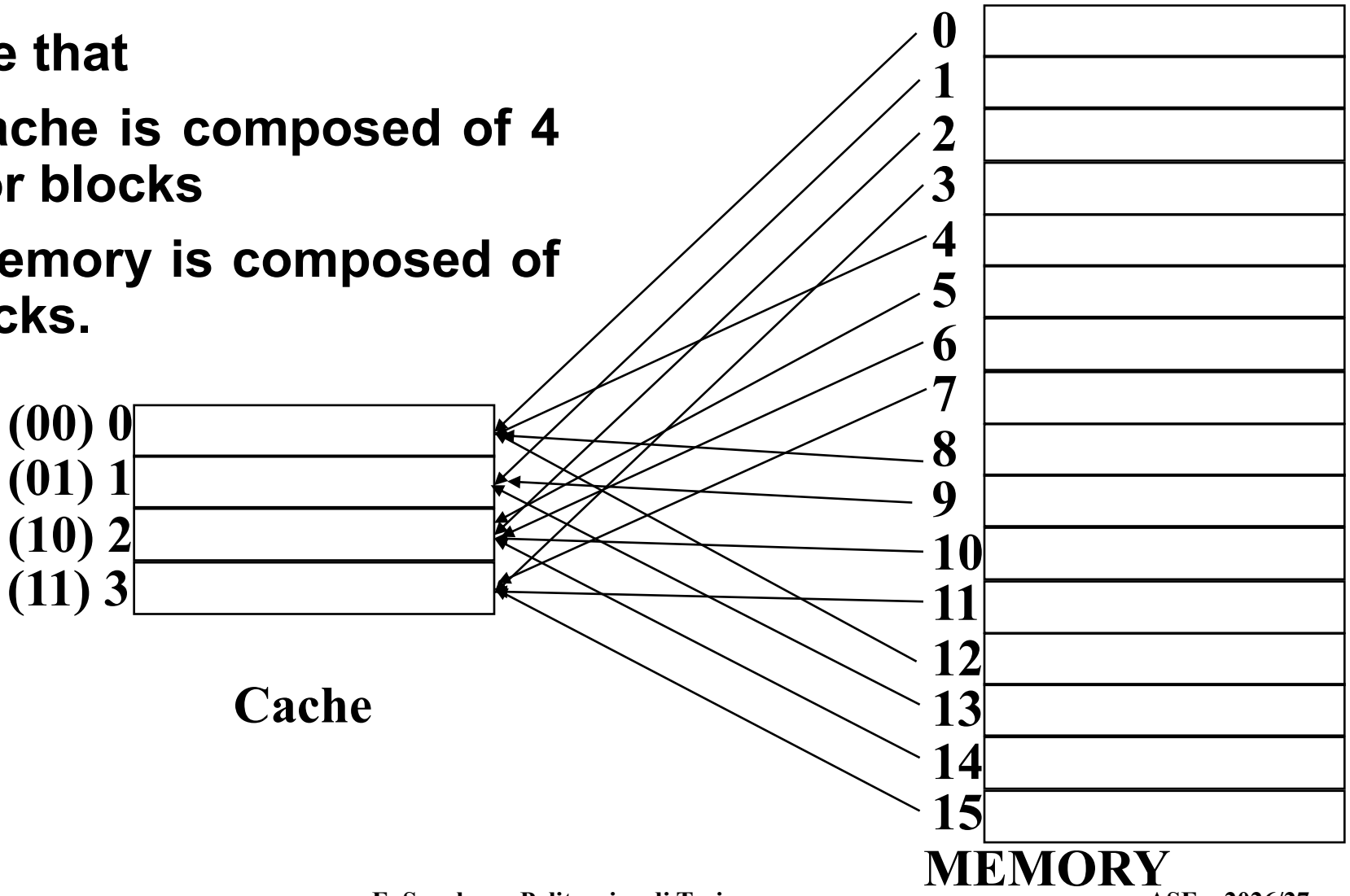
where *N* is the number of lines or blocks in the cache.

The computation of *k* can be easily performed by just taking the least significant bits in the value *i*.

Direct Mapping: example

Let assume that

- The cache is composed of 4 lines or blocks
- The memory is composed of 16 blocks.



Direct Mapping

Advantages:

- **the mechanism can be easily implemented in hardware (the least significant block identifier bits identify the cache line)**

Disadvantages:

- **if the program frequently accesses 2 blocks corresponding to the same cache line, a miss occurs at each memory access.**

Set Associative Mapping

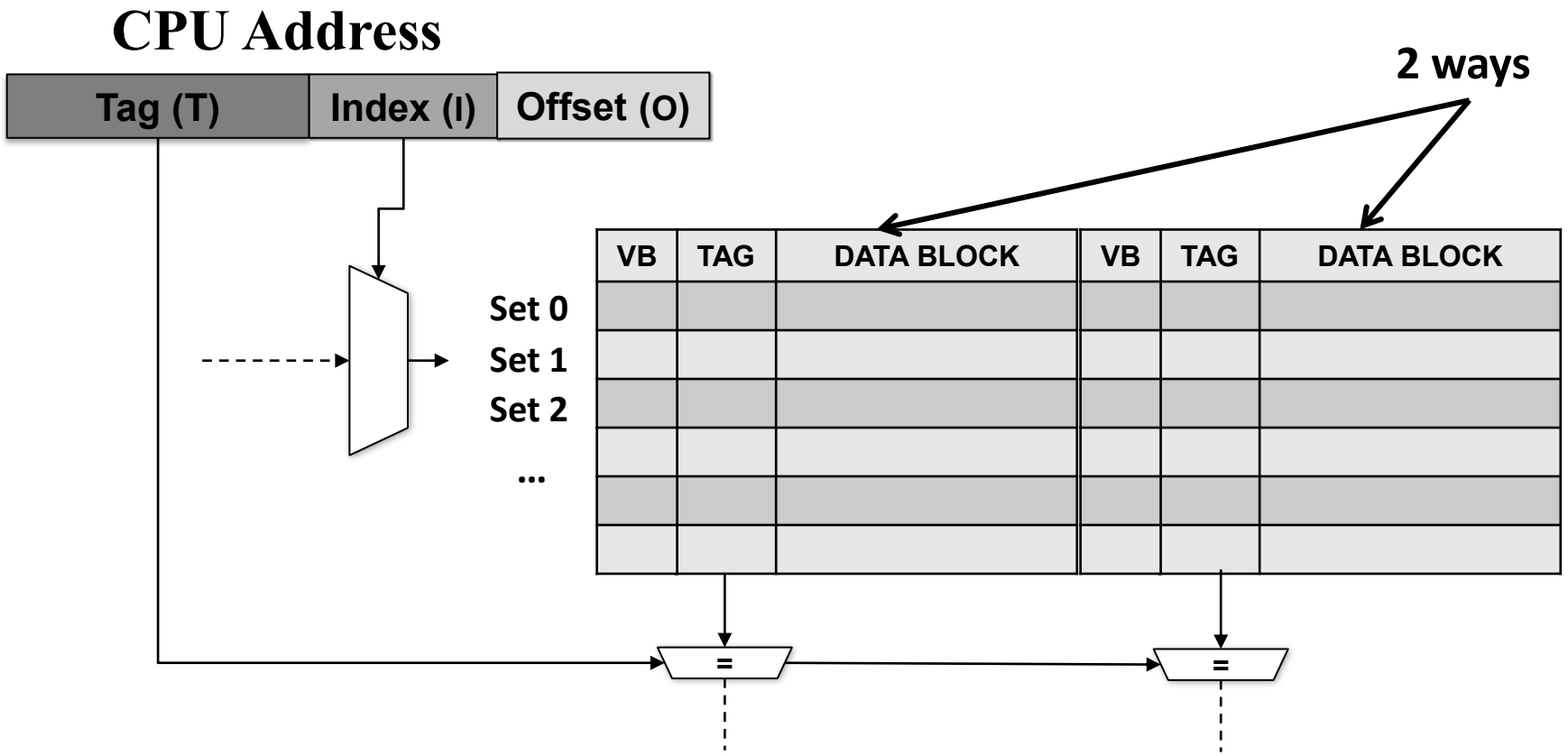
Characteristics:

- **the cache lines or blocks are subdivided into *S* sets, each consisting of *W (ways)* lines**
  - **S = N / W where N is the number of cache lines or blocks**
- **a memory block *i* is associated with the set *k*, with**
  - ***k = i mod S***
- **the block *i* can be put into any of the *W* lines of the set k.**

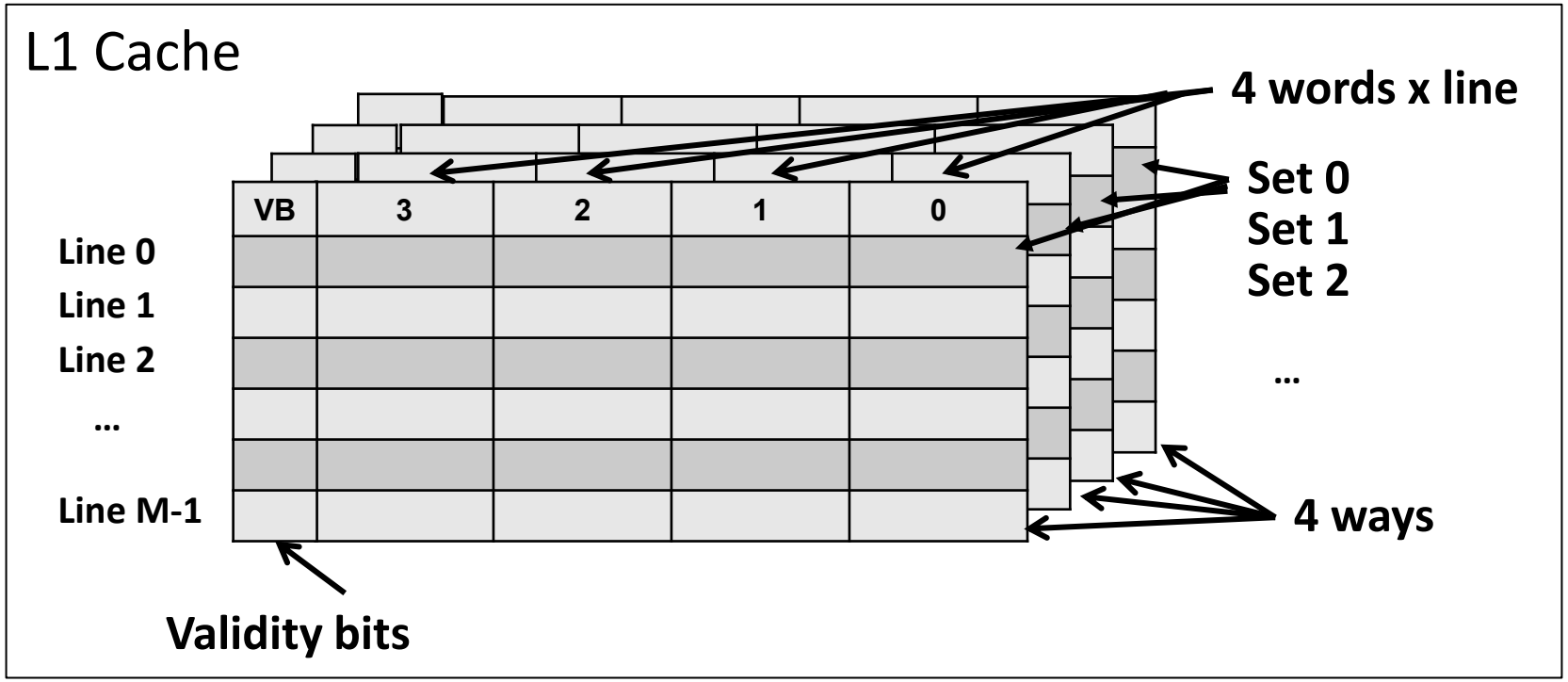
A set associative cache with *W* lines in each set is called a *W-ways* cache.

Common values for *W* are 2 and 4.

Set Associative Mapping



Set Associative Mapping



Fully Associative Mapping

Each block of the main memory can be stored in any cache block.

Advantages:

- maximum flexibility in choosing the cache block to use

Disadvantages:

- complexity of search hardware (usually adopting associative memory).

Effects of associativity

Effect of the cache associativity level on the address bits



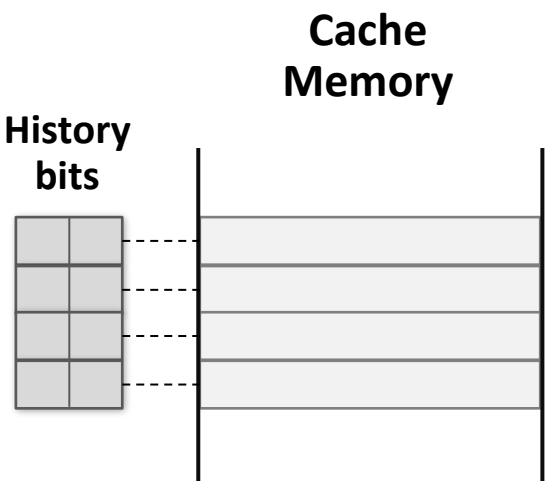
Replacing algorithm

It defines which cache line should be used to store a memory block, amongst those associated with the block (in the case of associative or set associative mapping).

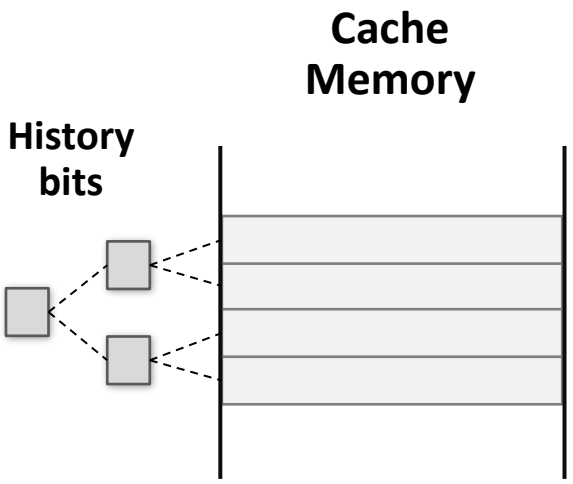
It is chosen from:

- *LRU* (Least Recently Used): the most used
- *FIFO* (First-In First-Out): the cheapest
- *LFU* (Least Frequently Used): theoretically, the most effective
- *random*: simple and effective.

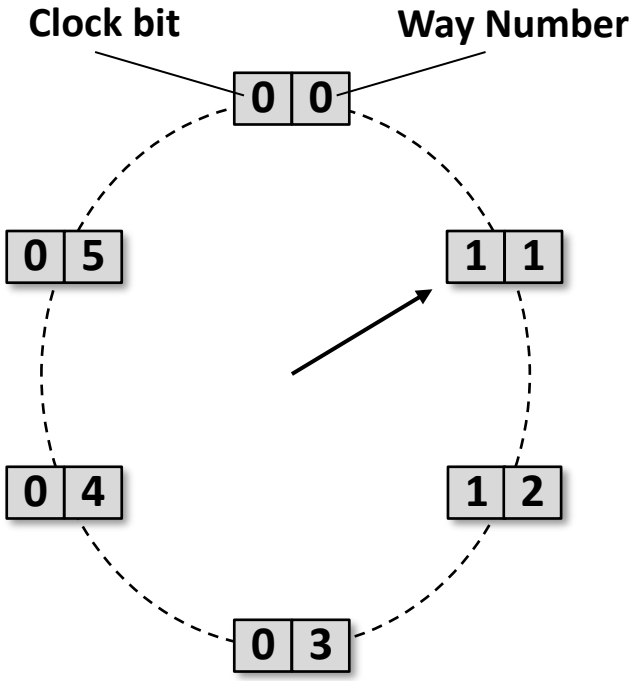
Replacement algorithm (II)



4-way
LRU



4-way
pseudo-
LRU

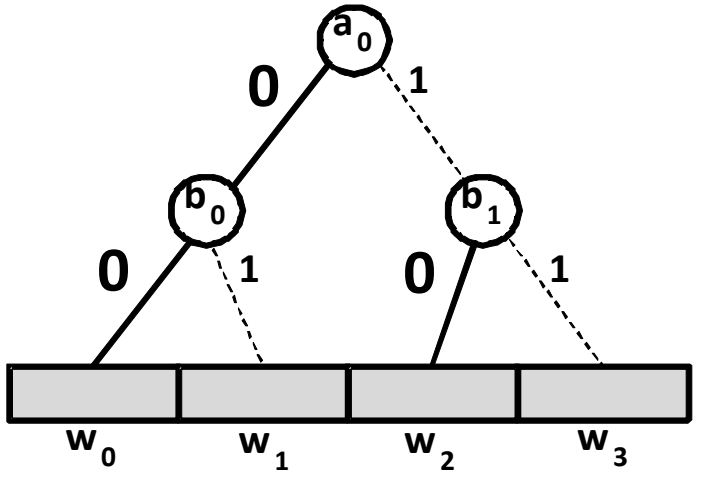


Clock
(FIFO-SC)

Example: pLRU replacement algorithm

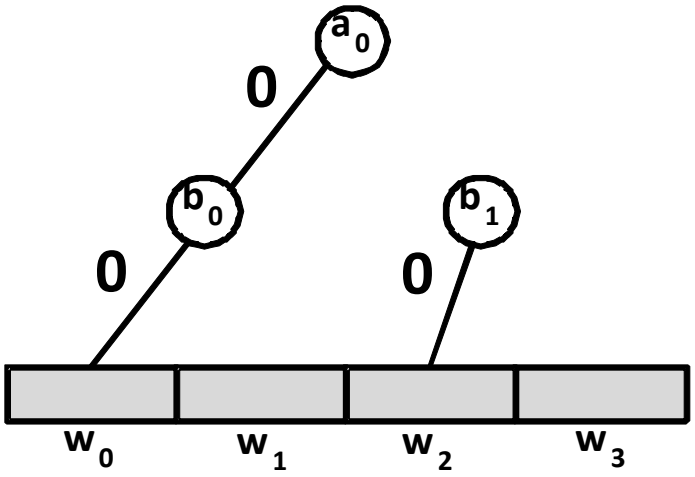
pLRU is an efficient approximation of a LRU algorithm

- The *age* of the cache ways is arranged in a binary tree
- Every node represents a *history bit*
- Access Order for a Way: AOW_x
  - $AOW_0 = a_0 + b_0$
  - $AOW_1 = a_0 + \overline{b}_0$
  - $AOW_2 = \overline{a}_0 + b_1$
  - $AOW_3 = \overline{a}_0 + \overline{b}_1$



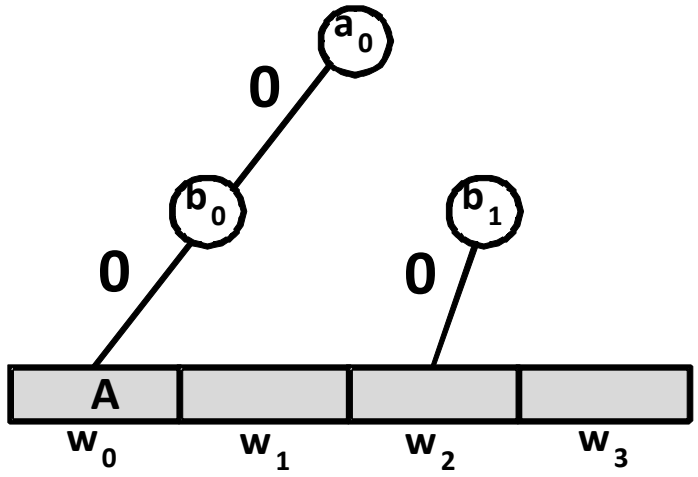
Example: pLRU replacement algorithm

Assume following data to be written: A B C D E



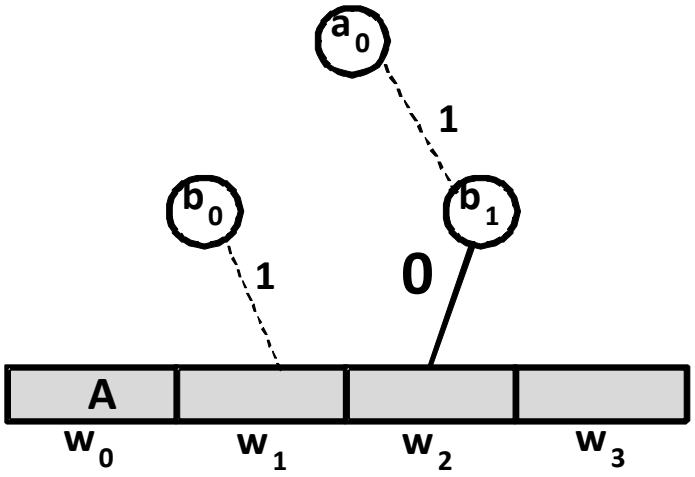
Example: pLRU replacement algorithm

Assume following data to be written: A B C D E



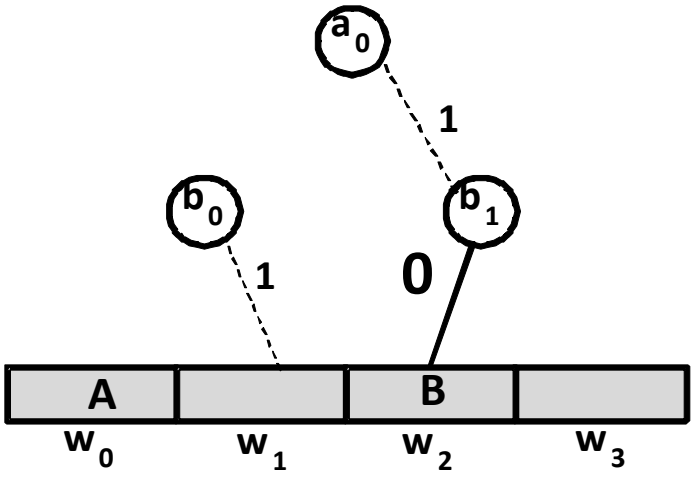
Example: pLRU replacement algorithm

Assume following data to be written: A B C D E



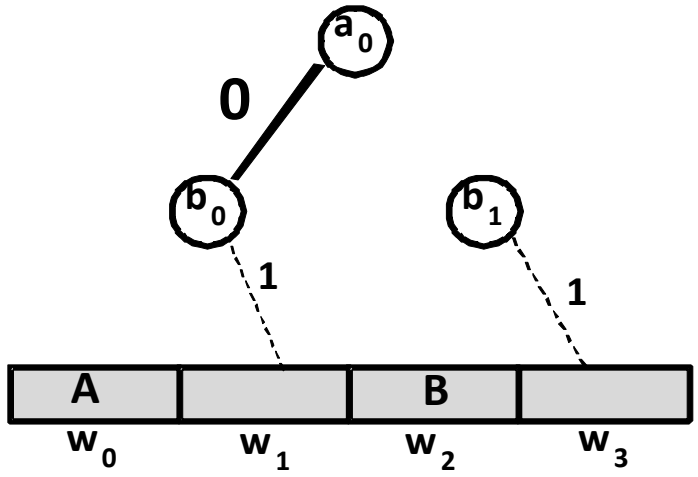
Example: pLRU replacement algorithm

Assume following data to be written: A B C D E



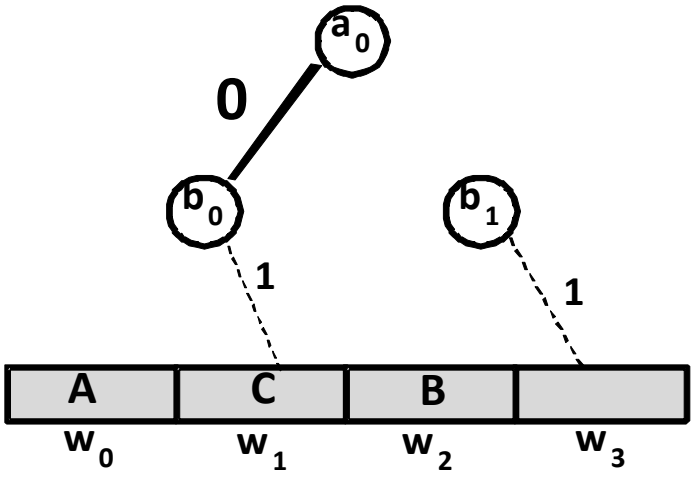
Example: pLRU replacement algorithm

Assume following data to be written: A B C D E



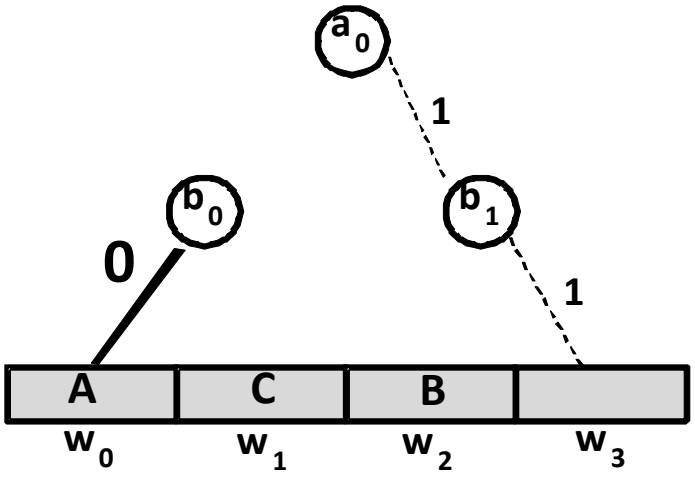
Example: pLRU replacement algorithm

Assume following data to be written: A B C D E



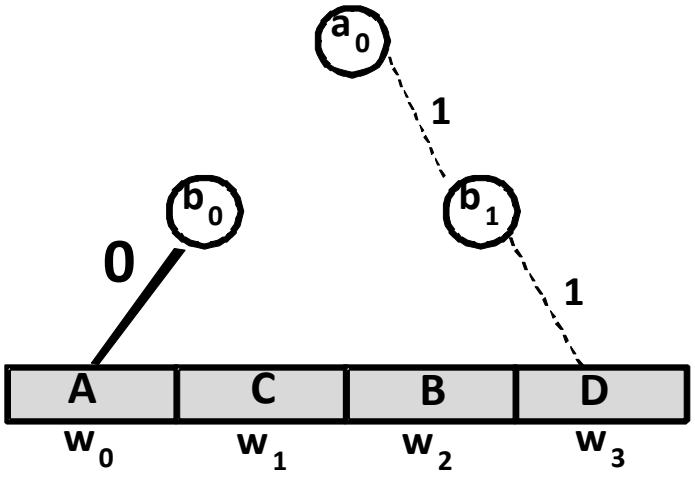
Example: pLRU replacement algorithm

Assume following data to be written: A B C D E



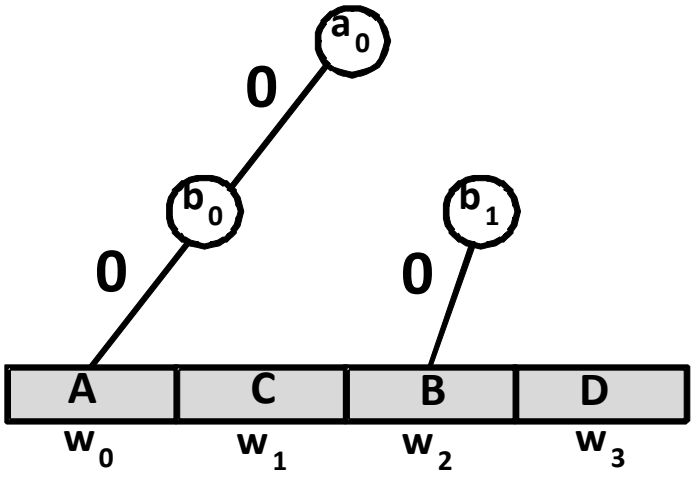
Example: pLRU replacement algorithm

Assume following data to be written: A B C D E



Example: pLRU replacement algorithm

Assume following data to be written: A B C D E



Main memory update

When a write operation is performed on a data in the cache, we also need to update the main memory.

Two solutions can be adopted:

- ***write-back***
- ***write-through.***

Write-Back

For each cache block, a flag (called *dirty bit*) is introduced, which remembers whether or not the block has been changed since it was loaded into the cache.

When a block is evicted from the cache and the dirty bit is set, the block is copied from the cache to the main memory.

Disadvantages:

- the replacement is slower because it sometimes requires copying in the main memory the replaced block
- in multiprocessor systems there may be incoherence between the caches of different processors
- it may not be possible to restore memory data after possible system failures.

Write-through

Each time the CPU performs a write operation, it writes on both the cache data and the main memory data.

The resulting loss of efficiency is limited by the fact that writing operations are usually much less numerous than reading ones.

Write-allocate

In the case of a write, and the data is not in the cache, there is a *write miss*.

Write-allocate or (fetch-on-write) loads the entire memory block in a cache line and performs the write operation in the cache.

It is typically used together with write-back, considering that data writes are buffered in the cache.

No write-allocate

In the case of a write miss, the cache does not load the memory block, sending the information directly to the memory.

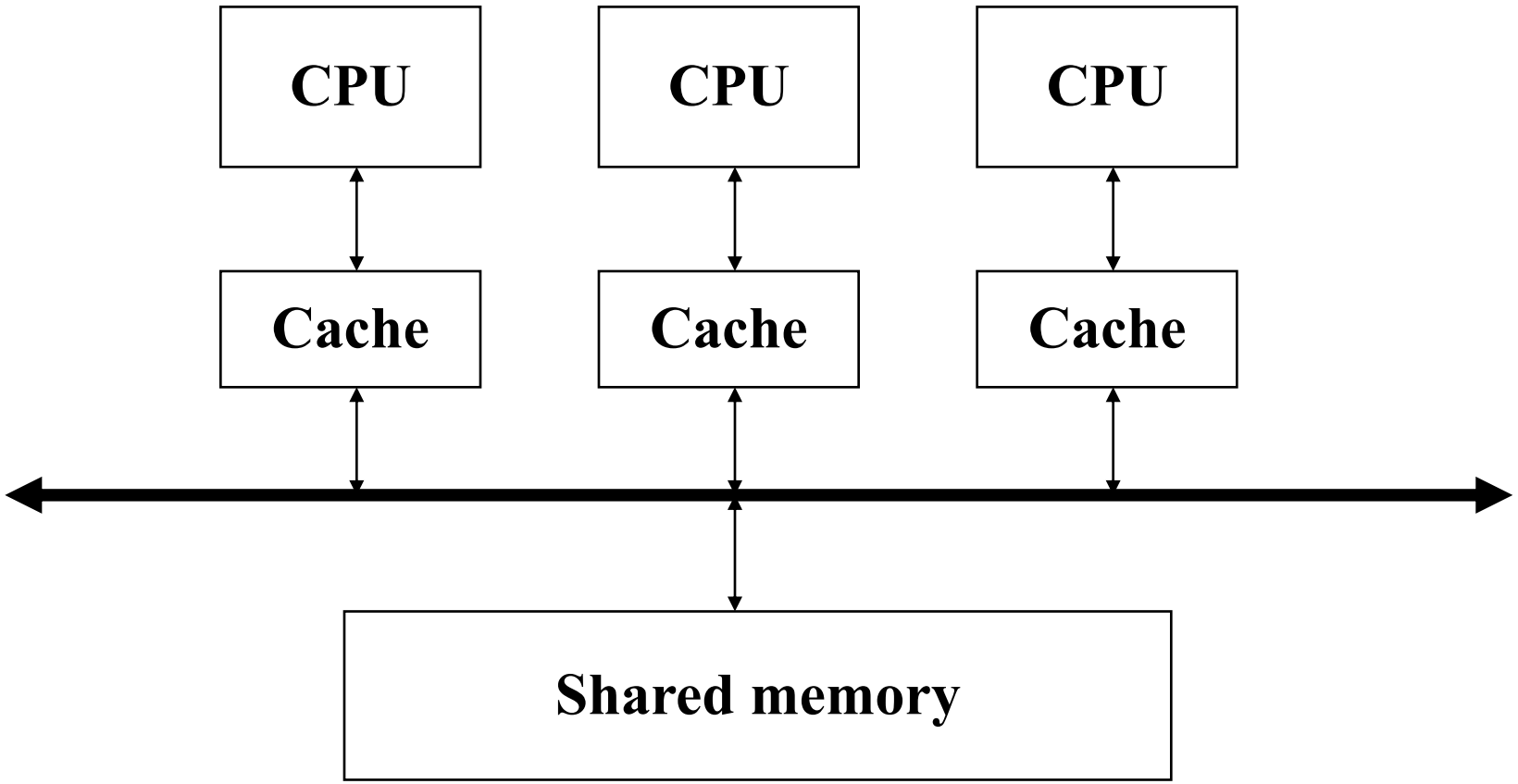
The data element results absent from the cache.

It is used together with write-through, considering that data writes go straight to the memory.

Cache coherence

This is a major problem in multiprocessor systems with shared memory, in which each processor has its own cache.

Similar problems may occur if the system uses a DMA controller.



Cache coherence - II

Coherence

- All reads by any processor must return the most recently written value
- Writes to the same location by any two processors are seen in the same order by all processors

Consistency

- If a processor writes location *A* followed by location *B*, any processor that sees the new value of *B* must also see the new value of *A.*

MESI protocol

A coherence protocol where every core/cache tracks sharing status of each line. In this case, more bits are needed for every line.

M *Modified*: The cache line was modified in this cache, and this cache is the only one having this value.

E *Exclusive:* This value is read from the memory but not yet modified. This cache is the only one having this value.

S *Shared*: This value is read from the memory but not yet modified. The value was read by other processors.

I *Invalid*: the line is invalid since modified externally. A shared value that is modified by a different processor.

First, second, third level caches

It may be convenient to have multiple levels of caches:

- a first level cache (L1), smaller and faster
- a second level cache (L2), slower but larger
- a third level cache (L3), even slower and bigger.

First, second, third level caches: behavior

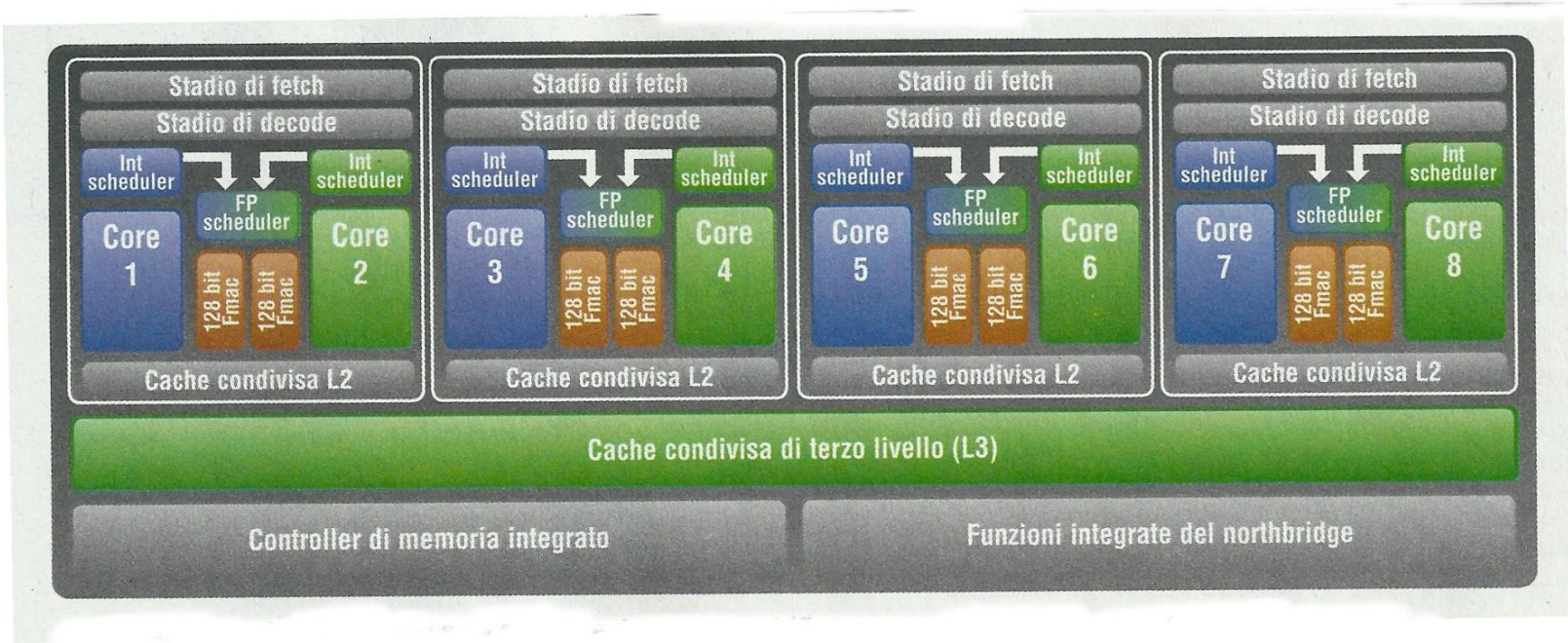
Each time the processor performs a memory access

- It checks first whether the word is in L1
- If so, it can access the word in L1
- If not, it checks whether the word is in L2
  - If so, it will access to the word in L2 and eventually update L1
  - If not, it checks whether the word is in L3
    - If yes, L3 is accessed and L2 is updated
    - If not, it accesses to the main memory and eventually updates L3.

Example: AMD Zambezi

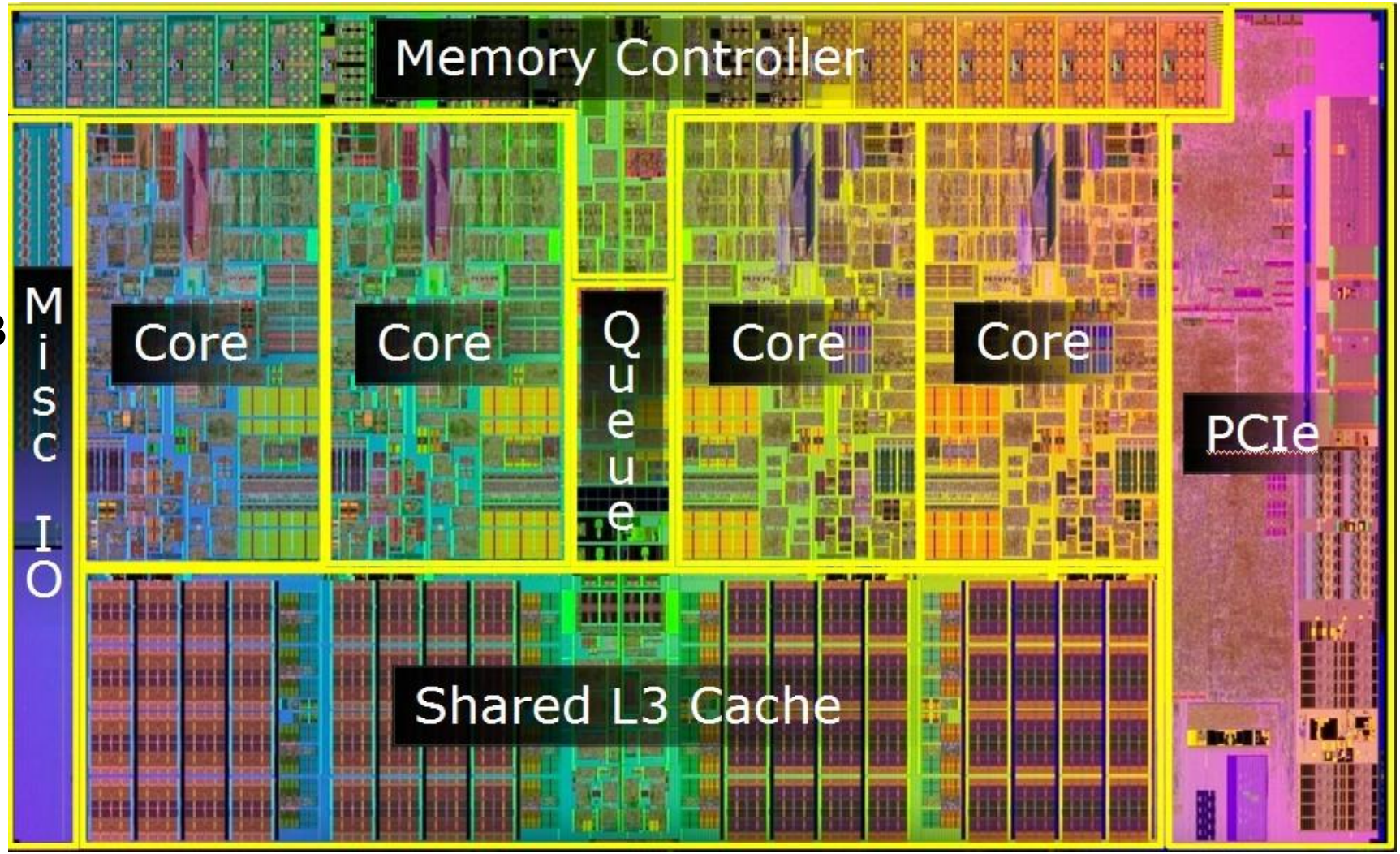
Zambezi is the high end CPU of the AMD Fusion family: it includes up to 8 cores, each equipped with its first level cache (L1) of 192 kB.

Each core pair uses a second level (L2) cache of 2 or 4 MBytes. The cores of the same device share the third level cache (L3) of 8 MBytes.



Example: Intel Lynnfield

L1= 64 kB
L2= 256 kB
L3= 8MB



Example of cache size

Let us consider a cache with the following characteristics:

- 64 kByte size (data only)
- direct mapping
- 4 bytes blocks in every line of the cache and memory
- 32 bits addresses.

You are asked to determine the structure of the cache (number of lines, size of the tag field).

Example of cache size

Since each block is compose of 4 bytes, the Memory is composed of 2^{30} lines or blocks.

The number of lines in the cache is 64kB / 4 = 16kB =2^{14}.

The tag field identifies the block currently stored in each line.

Hence, the tag field should be composed of 30 bit.

However, since in the generic cache line only blocks whose index has the 14 least significant bits equal to the line index, the tag field is composed of 16 bits, only.

Hence, the total size of the cache is given by:

$$2^{14} \times (32 + 16) = 2^{14} \times 48 = 768\text{kbit} = 96\text{kByte}$$

Example 2

It is given a system composed by the following memory organization

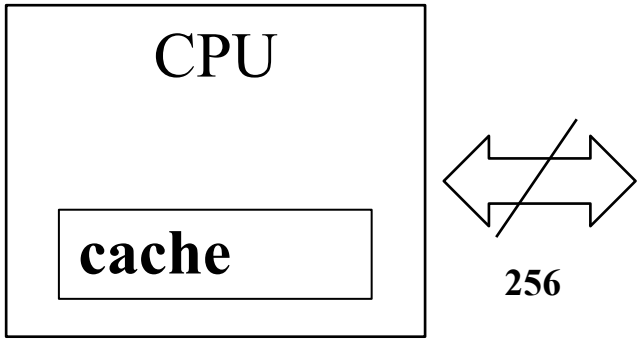
- A Central Data memory, located in many Flash cores
  - With non-contiguous addressing (see the illustration)
  - Every Flash can emit up to 256 bits (32 bytes) per read
- A cache memory of 4KB (data only), organized as 2-ways set associative
- CPU can fetch 64 bits (8 bytes), but cache lines contain 256 bits = 32 bytes
- Addresses are expressed on 32 bits

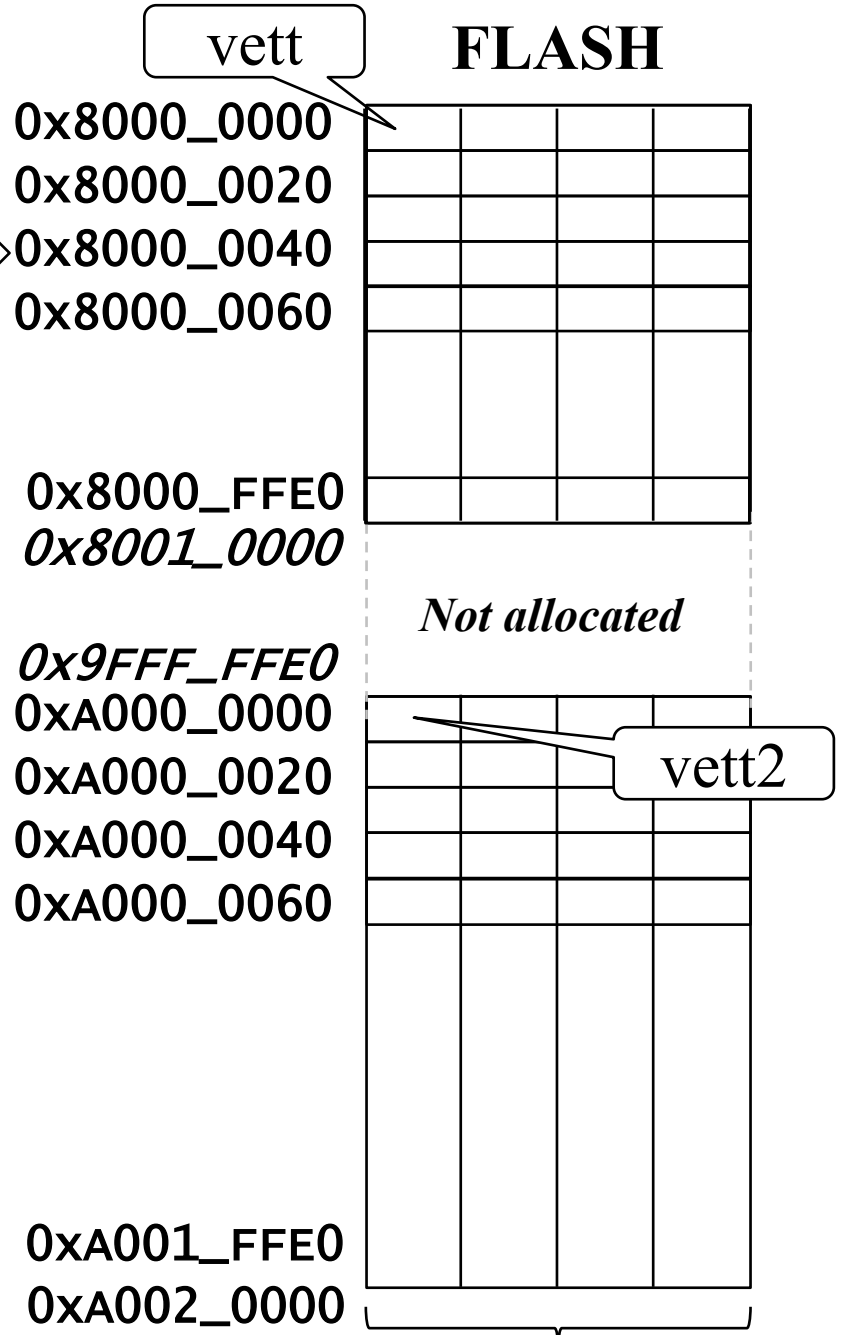
Determine the size of tag, index and offset that better fit the memory organization.

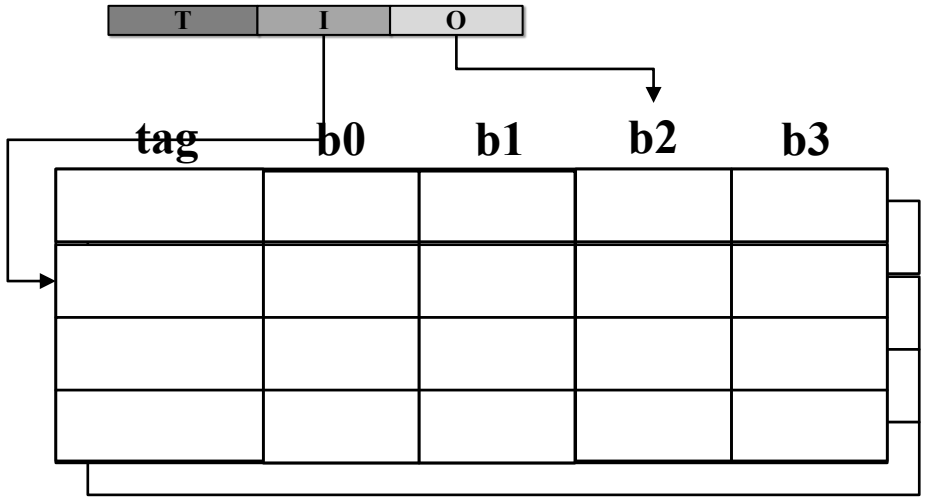
Determine the overall size of the cache memory.

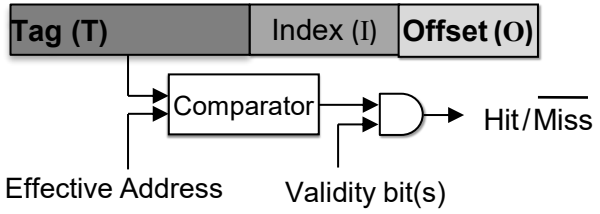
```
      add r2, r0, N
loop:
      ld r1, vett(r2)
      add r2, r2, 8
      bnez r2, loop
 …….

     add r2, r0, M
loop2:
      ld r1, vett2(r2)
      sub r2, r2, 8
      bnez r2, loop2
```



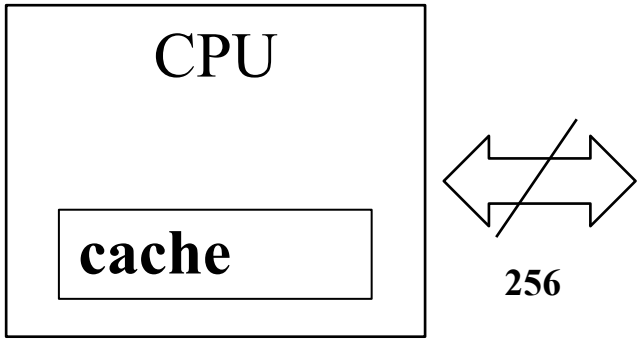


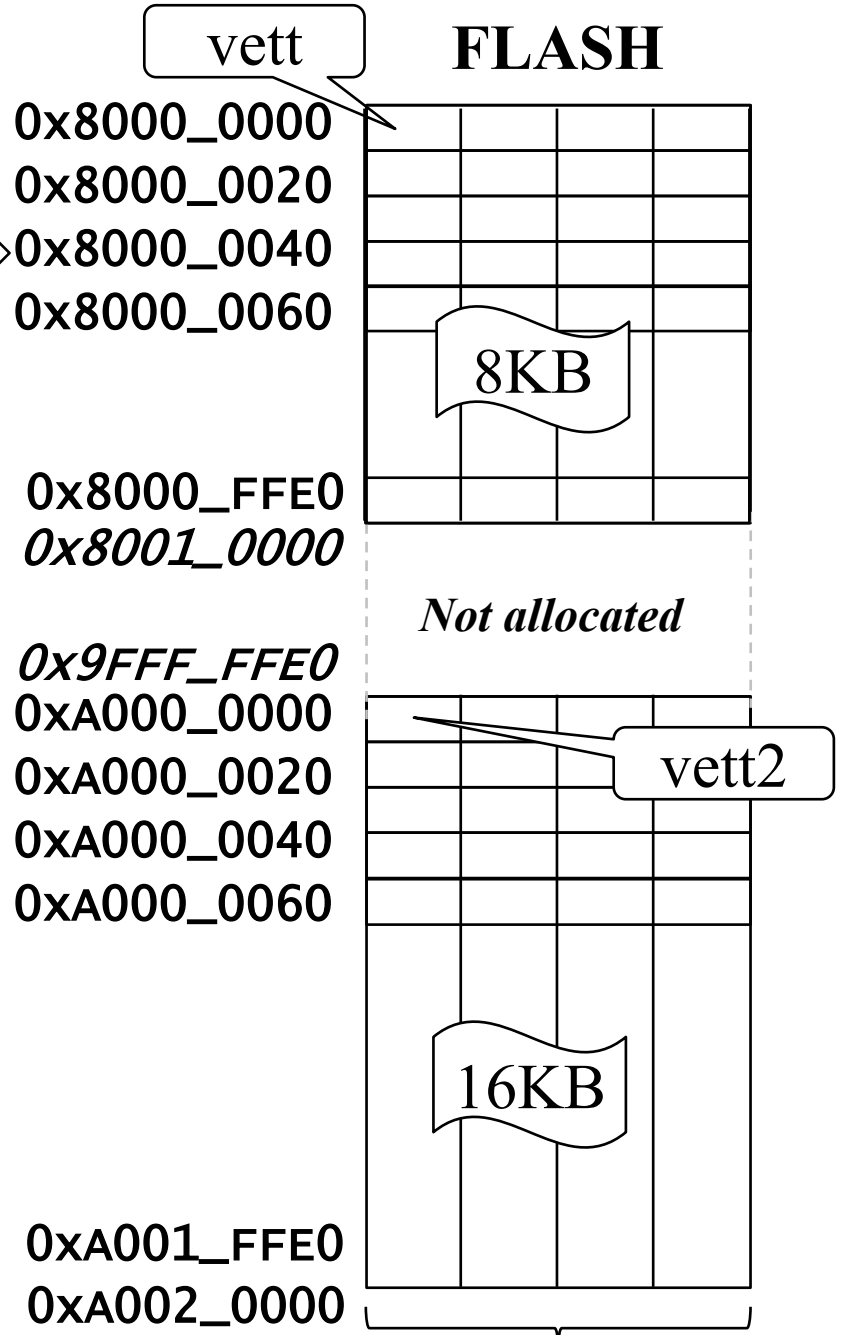


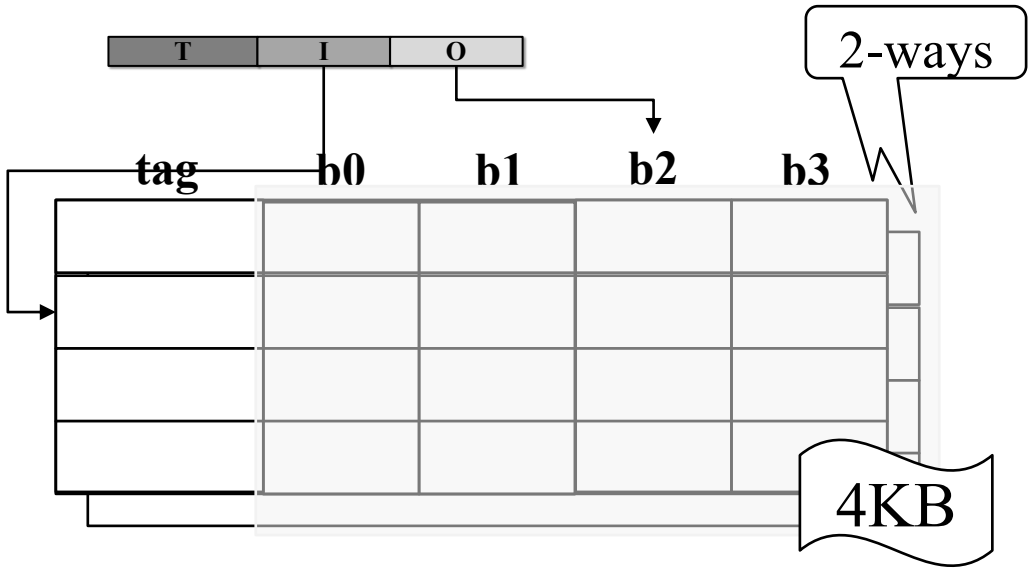


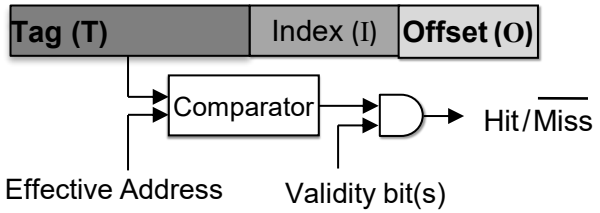
```
      add r2, r0, N
loop:
      ld r1, vett(r2)
      add r2, r2, 8
      bnez r2, loop
 …….

     add r2, r0, M
loop2:
      ld r1, vett2(r2)
      sub r2, r2, 8
      bnez r2, loop2
```



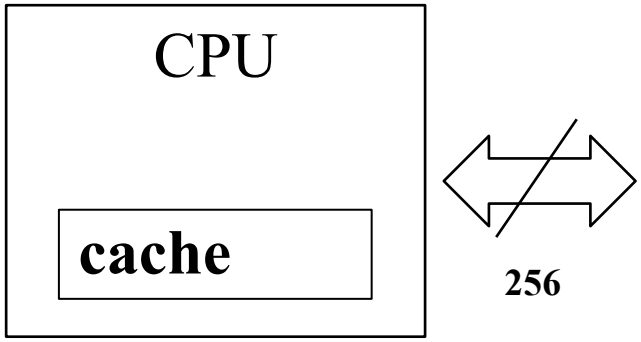


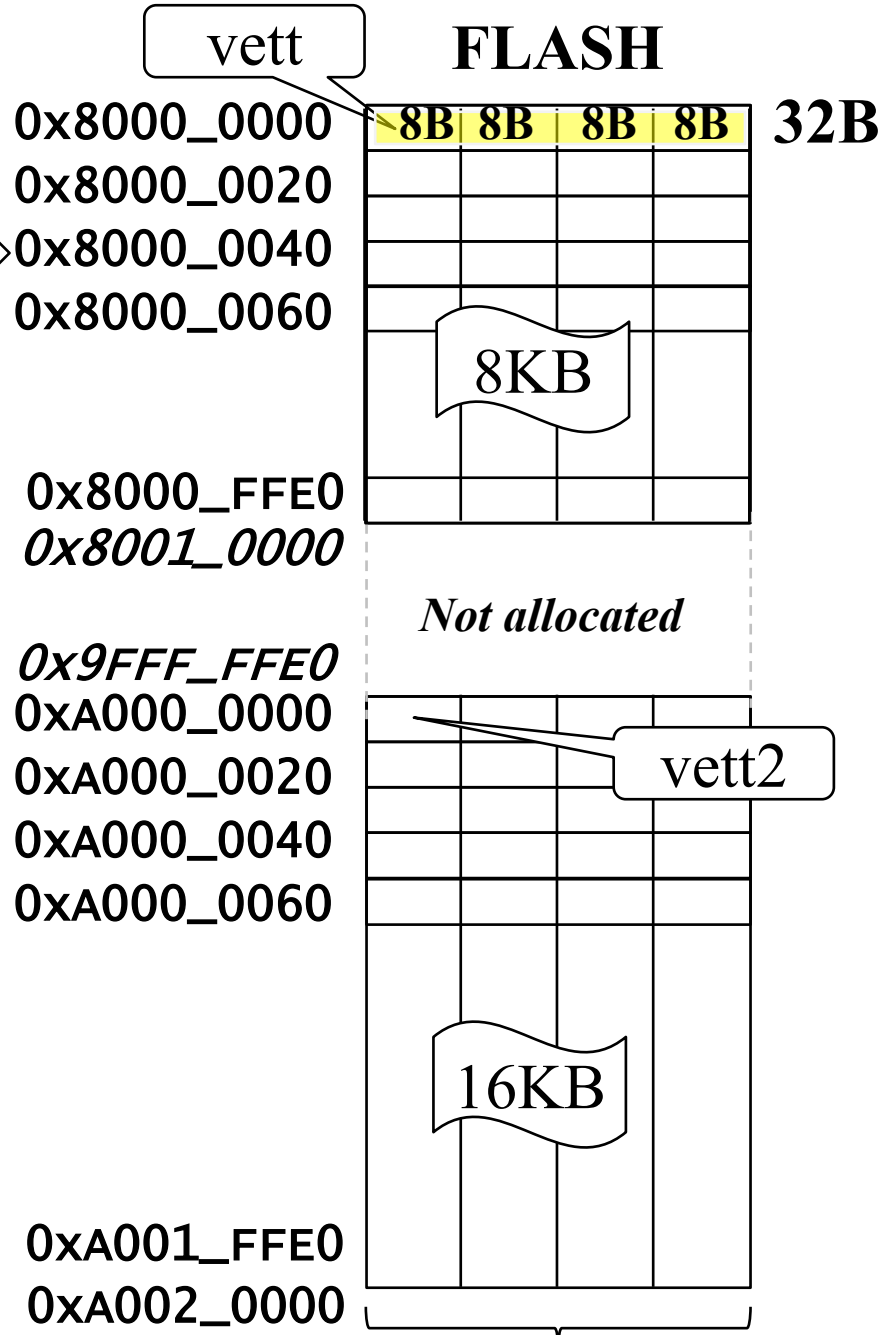


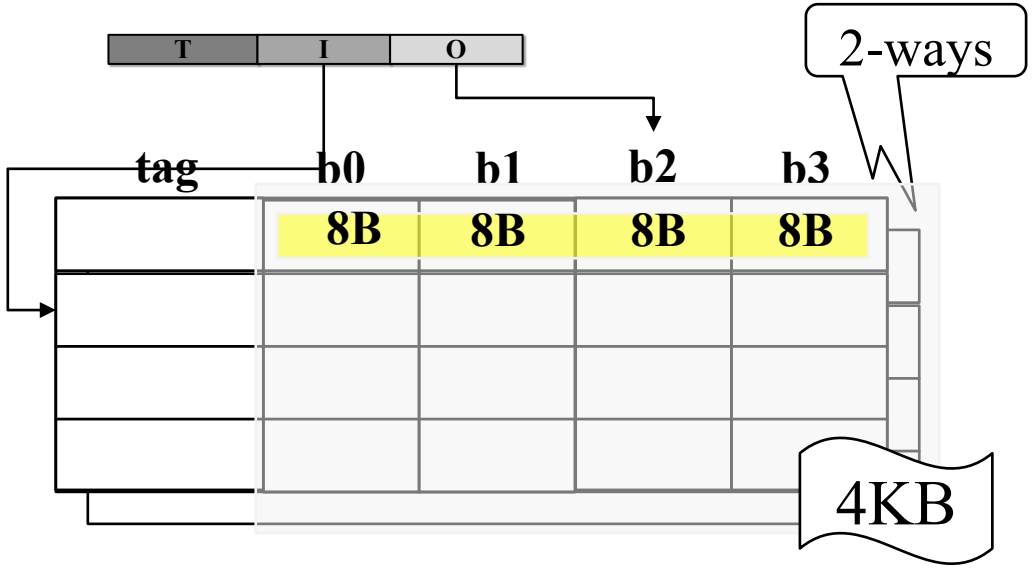


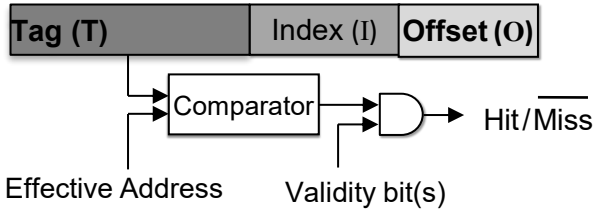
```
      add r2, r0, N
loop:
      ld r1, vett(r2)
      add r2, r2, 8
      bnez r2, loop
 …….

     add r2, r0, M
loop2:
      ld r1, vett2(r2)
      sub r2, r2, 8
      bnez r2, loop2
```



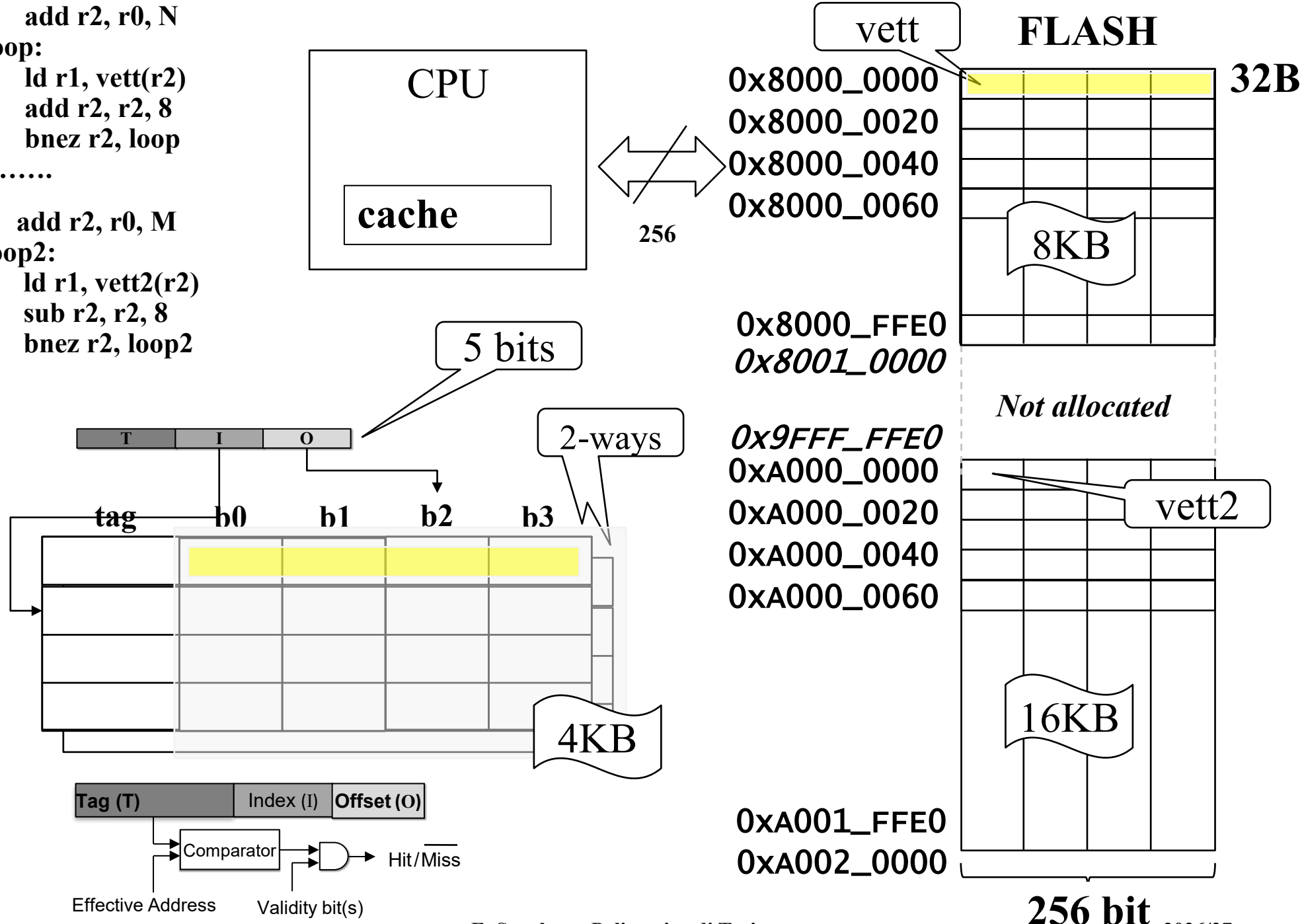






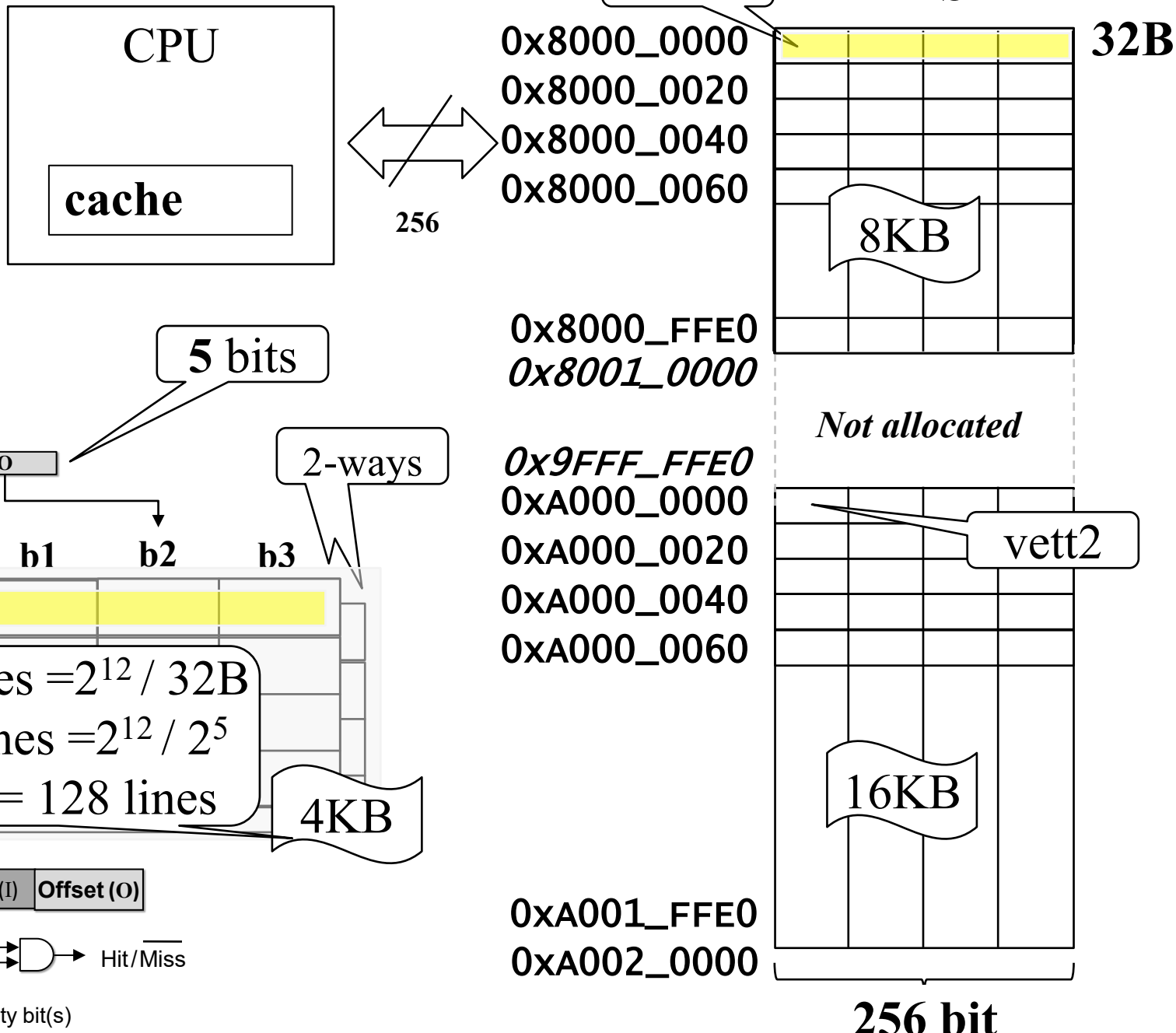
```
      add r2, r0, N
loop:
      ld r1, vett(r2)
      add r2, r2, 8
      bnez r2, loop
 …….

     add r2, r0, M
loop2:
      ld r1, vett2(r2)
      sub r2, r2, 8
      bnez r2, loop2
```



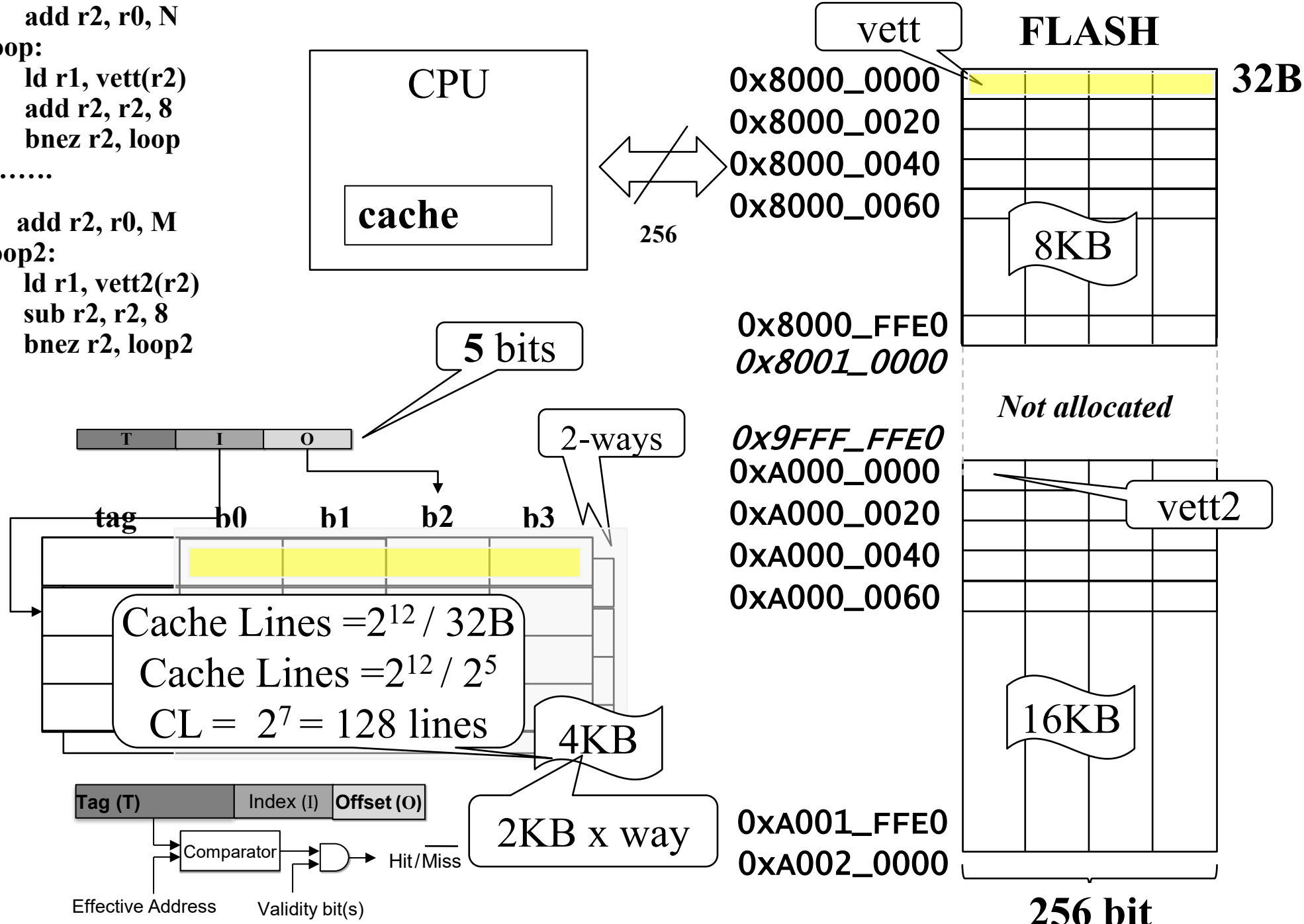
```
      add r2, r0, N
loop:
      ld r1, vett(r2)
      add r2, r2, 8
      bnez r2, loop
 …….

     add r2, r0, M
loop2:
      ld r1, vett2(r2)
      sub r2, r2, 8
      bnez r2, loop2
```



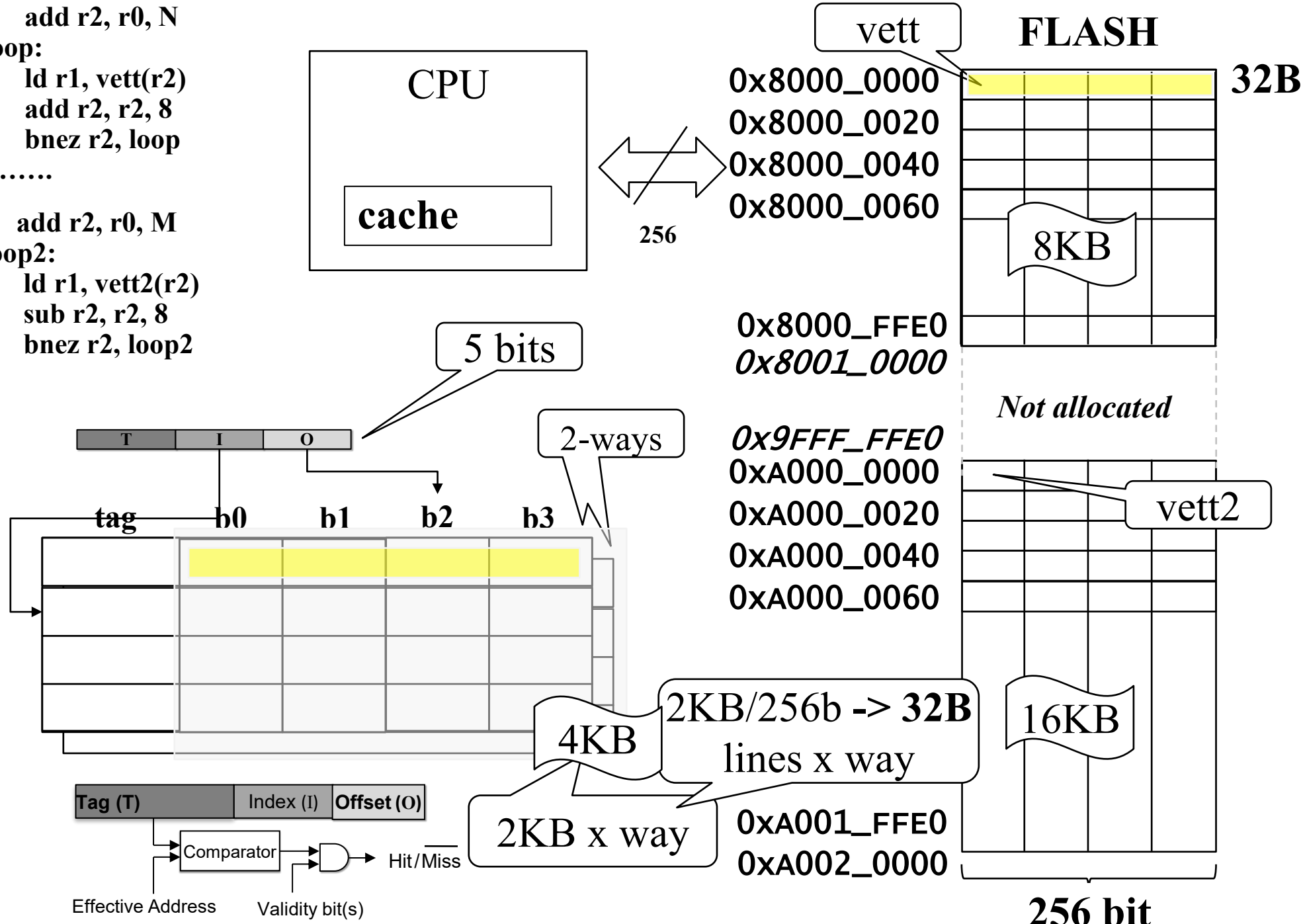
```
      add r2, r0, N
loop:
      ld r1, vett(r2)
      add r2, r2, 8
      bnez r2, loop
 …….

     add r2, r0, M
loop2:
      ld r1, vett2(r2)
      sub r2, r2, 8
      bnez r2, loop2
```



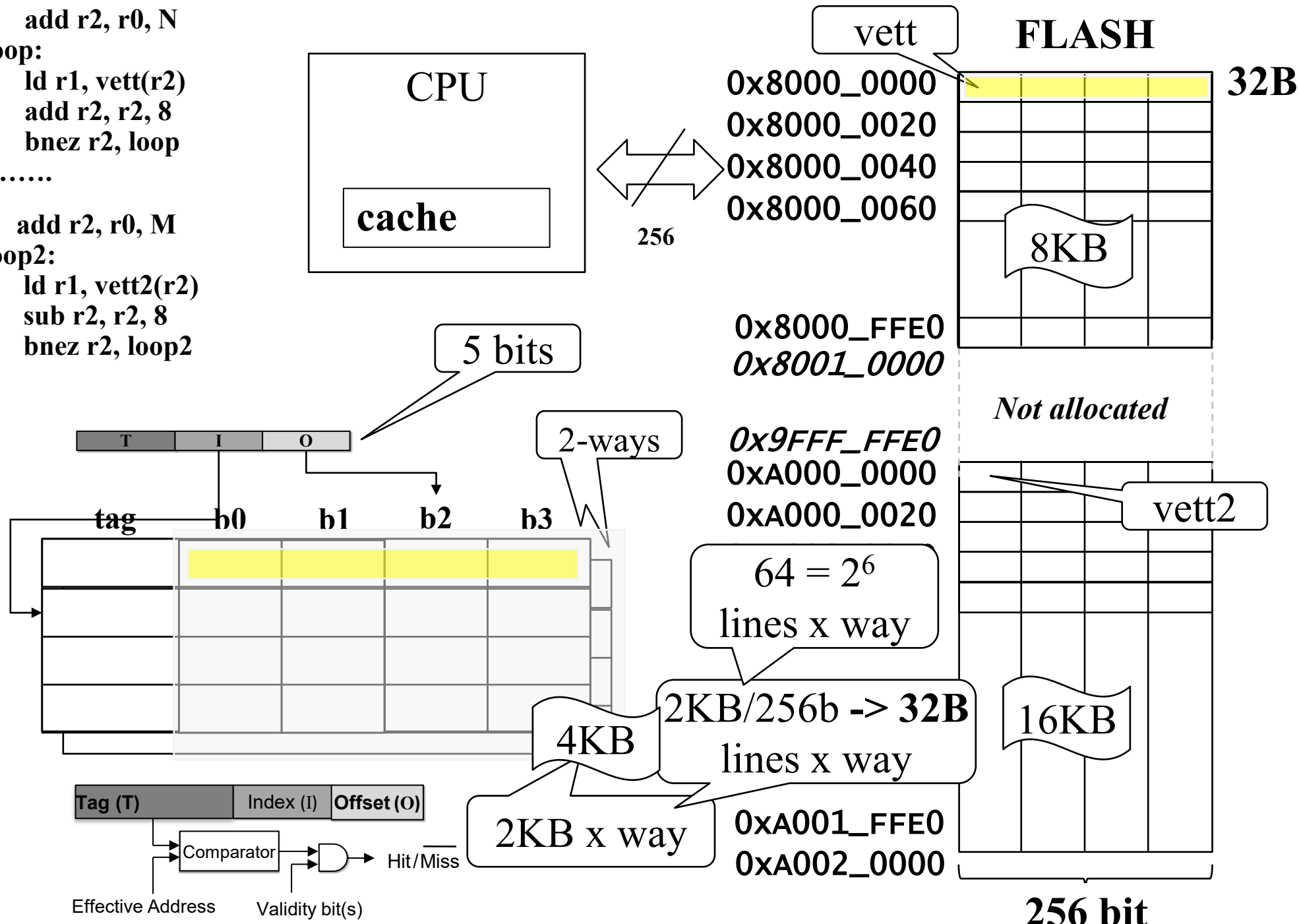
```
      add r2, r0, N
loop:
      ld r1, vett(r2)
      add r2, r2, 8
      bnez r2, loop
 …….

     add r2, r0, M
loop2:
      ld r1, vett2(r2)
      sub r2, r2, 8
      bnez r2, loop2
```



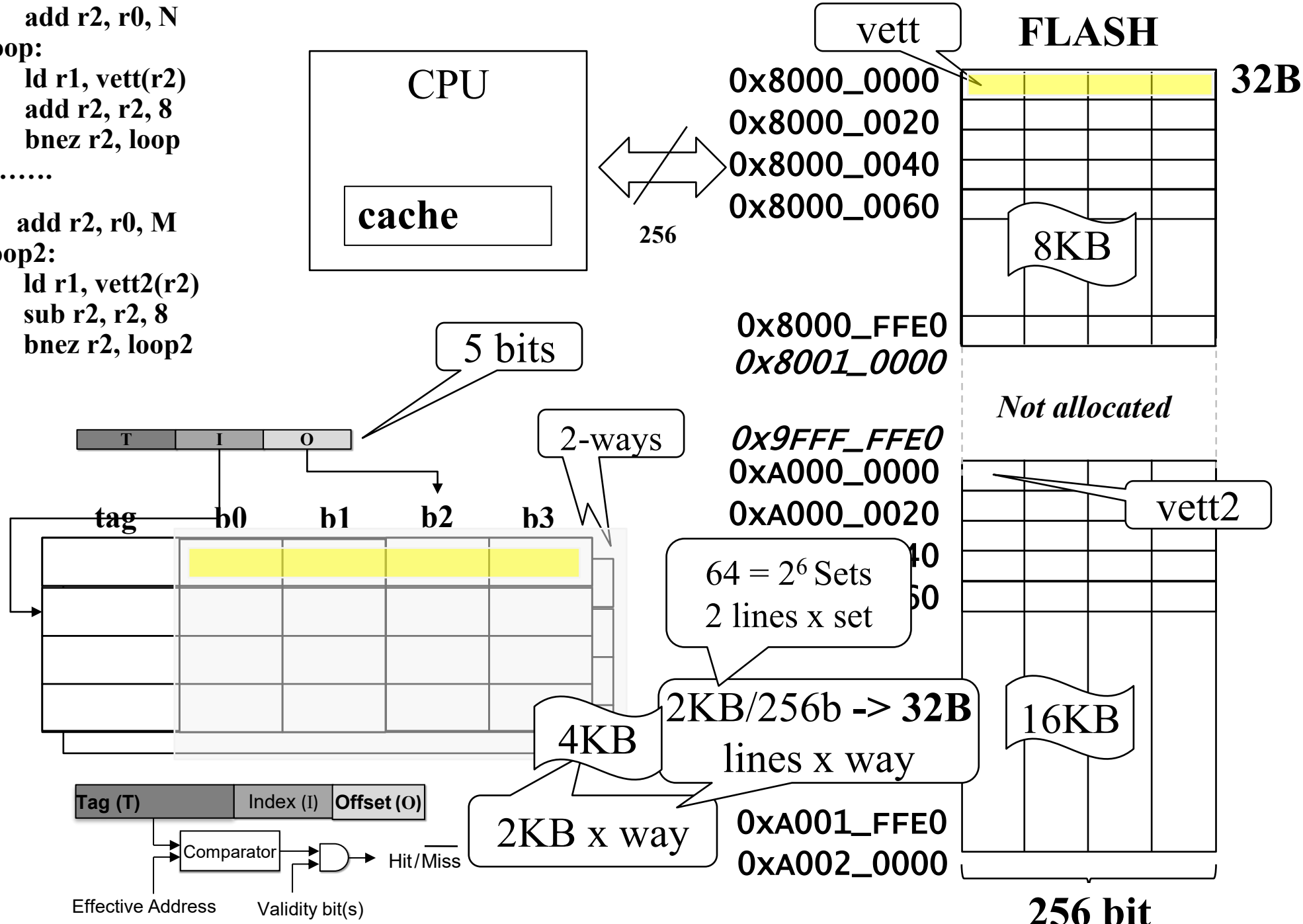
```
    add r2, r0, N
loop:
    ld r1, vett(r2)
    add r2, r2, 8
    bnez r2, loop
 …….

   add r2, r0, M
loop2:
    ld r1, vett2(r2)
    sub r2, r2, 8
    bnez r2, loop2
```



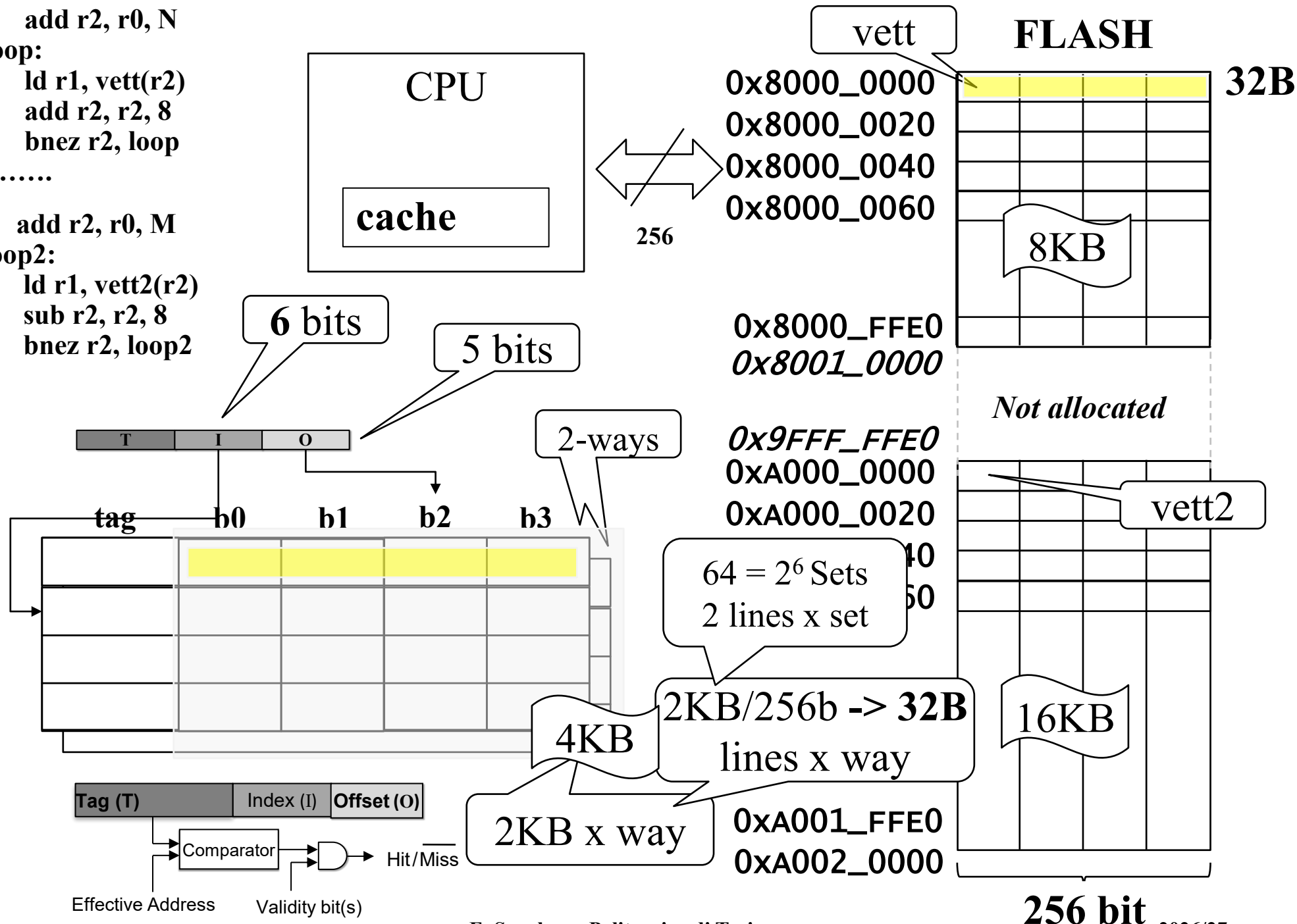
```
     add r2, r0, N
loop:
     ld r1, vett(r2)
     add r2, r2, 8
     bnez r2, loop
 …….

    add r2, r0, M
loop2:
     ld r1, vett2(r2)
     sub r2, r2, 8
     bnez r2, loop2
```



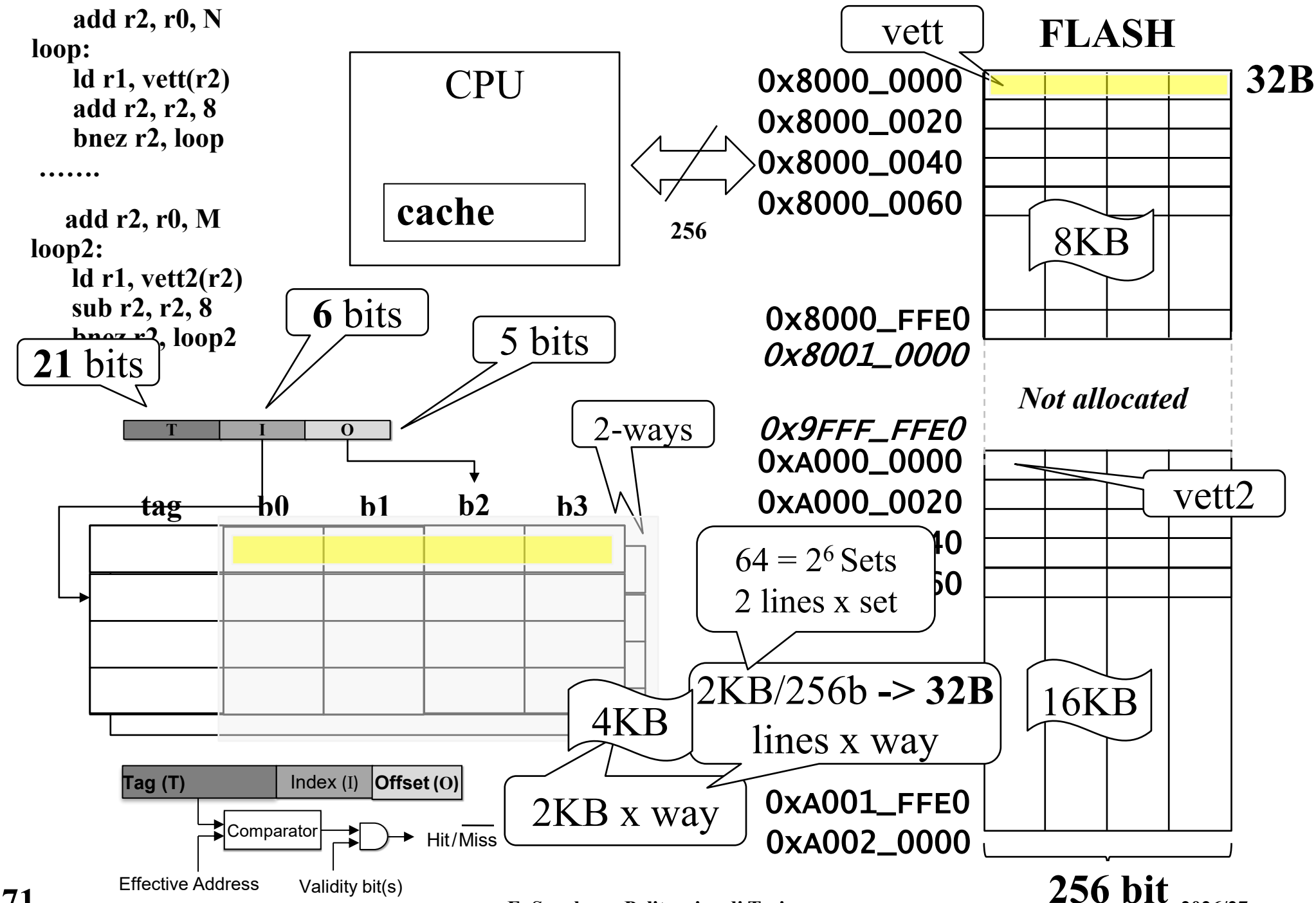
```
     add r2, r0, N
loop:
     ld r1, vett(r2)
     add r2, r2, 8
     bnez r2, loop
 …….

    add r2, r0, M
loop2:
     ld r1, vett2(r2)
     sub r2, r2, 8
     bnez r2, loop2
```



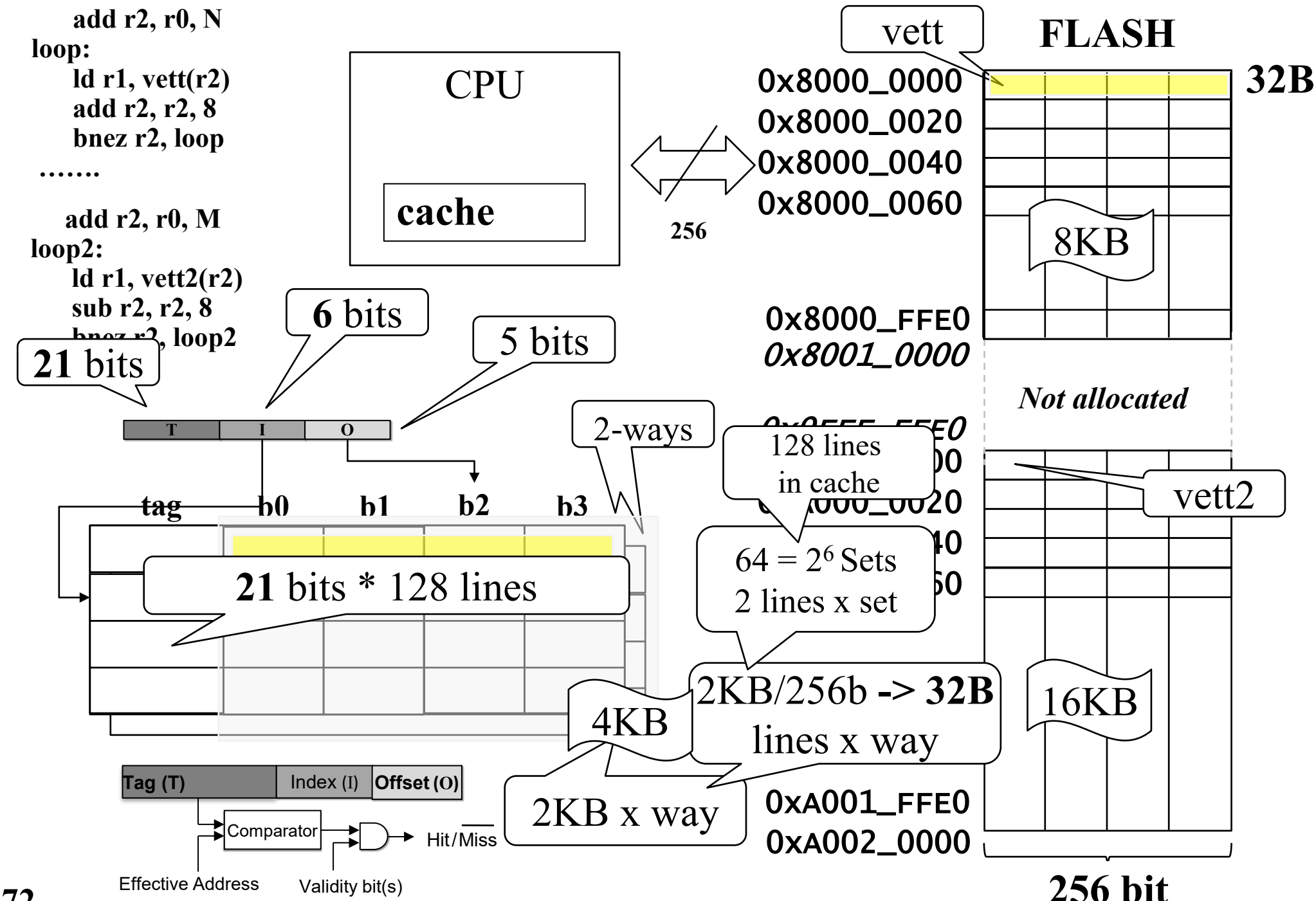
```
      add r2, r0, N
loop:
      ld r1, vett(r2)
      add r2, r2, 8
      bnez r2, loop
 …….

     add r2, r0, M
loop2:
      ld r1, vett2(r2)
      sub r2, r2, 8
      bnez r2, loop2
```



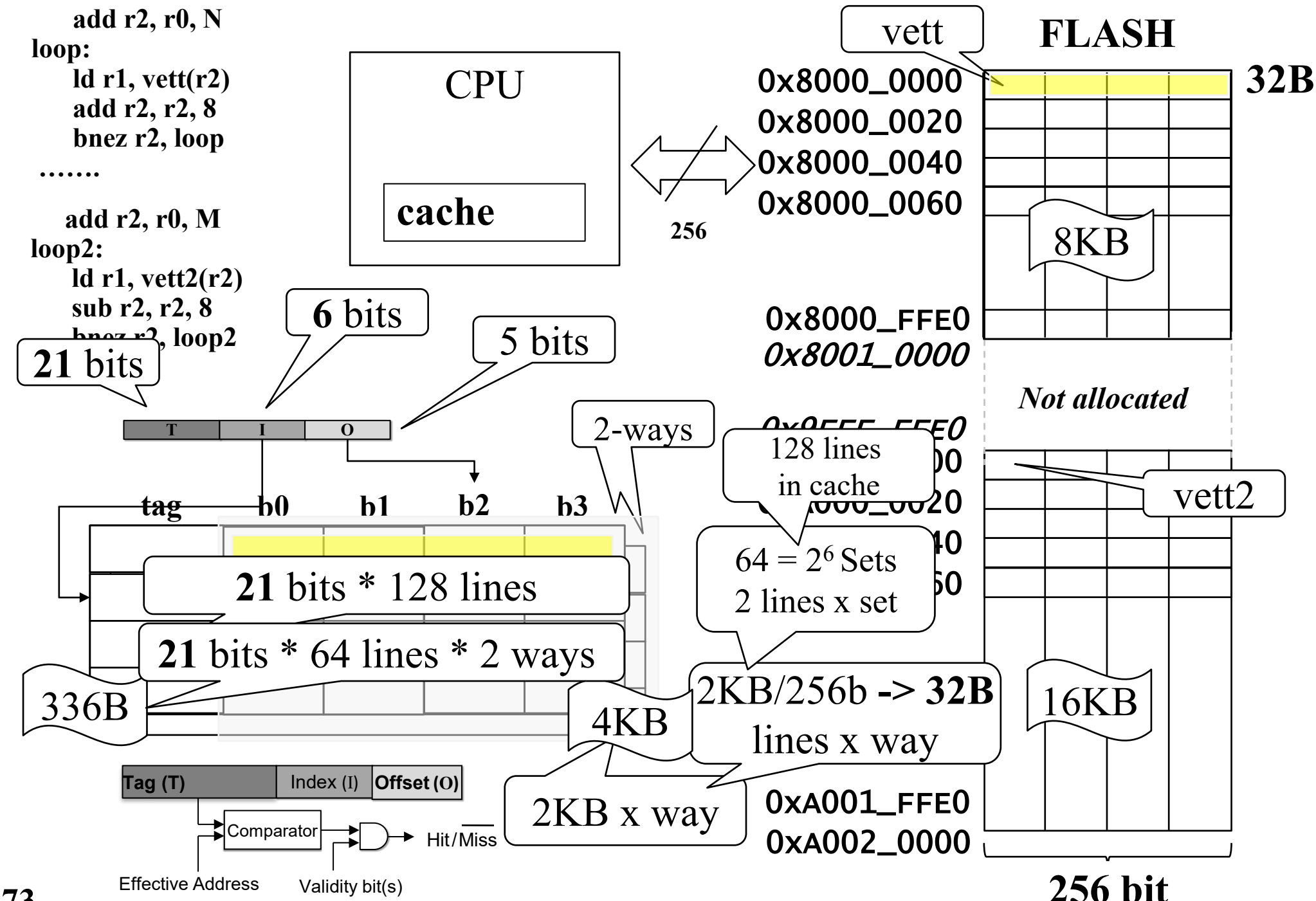
```
      add r2, r0, N
loop:
      ld r1, vett(r2)
      add r2, r2, 8
      bnez r2, loop
 …….

     add r2, r0, M
loop2:
      ld r1, vett2(r2)
      sub r2, r2, 8
      bnez r2, loop2
```



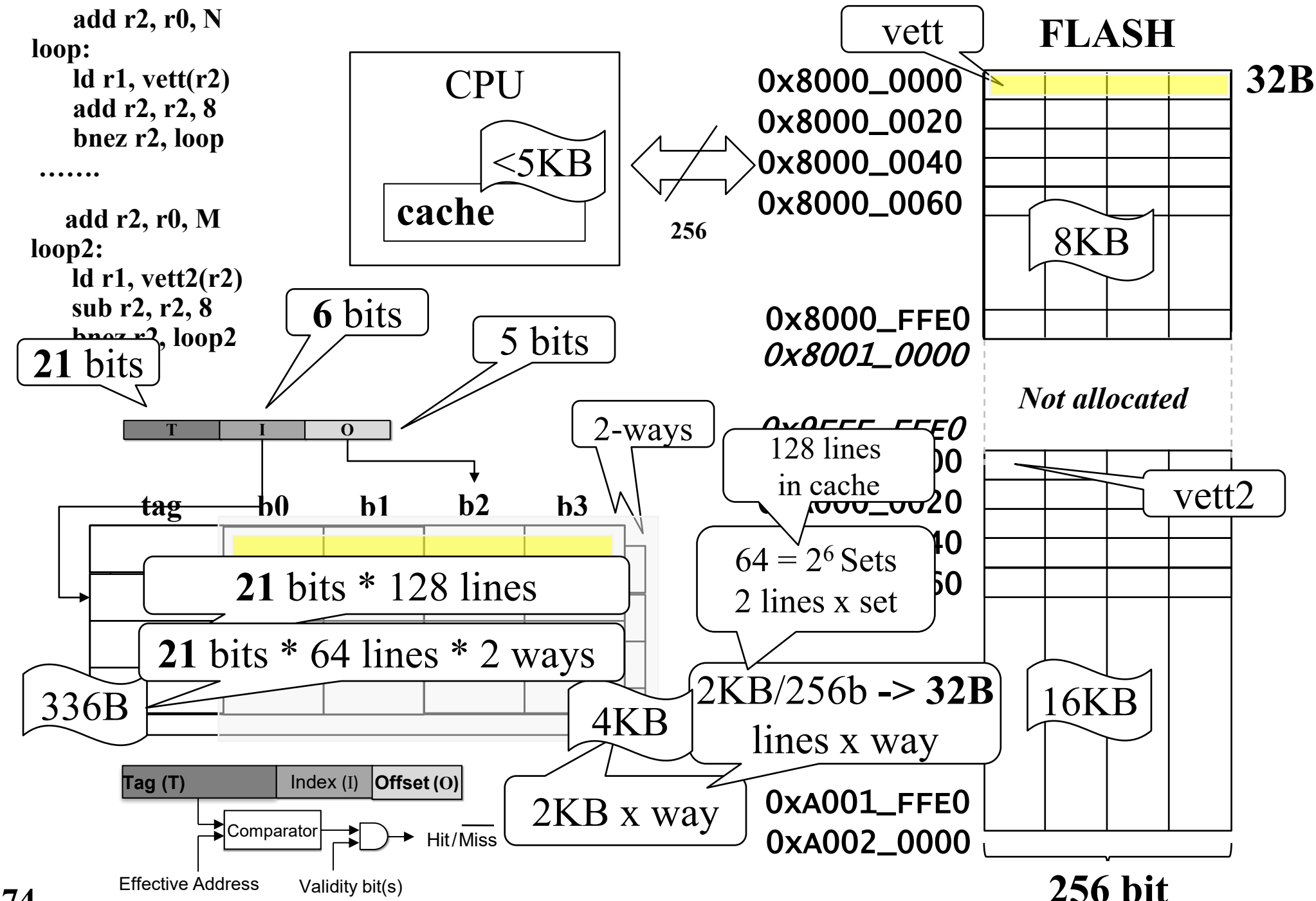
```
      add r2, r0, N
loop:
      ld r1, vett(r2)
      add r2, r2, 8
      bnez r2, loop
 …….

     add r2, r0, M
loop2:
      ld r1, vett2(r2)
      sub r2, r2, 8
      bnez r2, loop2
```



```
      add r2, r0, N
loop:
      ld r1, vett(r2)
      add r2, r2, 8
      bnez r2, loop
 …….

     add r2, r0, M
loop2:
      ld r1, vett2(r2)
      sub r2, r2, 8
      bnez r2, loop2
```



Example of cache size - 3

Determine the memory size of a Data Cache of 4KB (data only), organized as 4-ways set associative, and allocating in every line 32 bytes.

Memory addresses are expressed on 32 bits.

Determine the size of the cache parameters: tag, index and offset that fit the memory organization.

Determine the overall size of the cache memory.

Example of cache size - 3

Offset:

Every line contains 32 bytes, so 5 address bits are used

Offset = 5 bits (2^5 = 32)

Lines:

DataSize / #bytes x line

4KB / 32 = 2 x 2^12 / 2^5 = 2^7 lines

128 lines

Sets:

Numb. lines / Associativity = 2^7 / 4

= 2^5 sets

The index uses 5 address bits.

Tag:

32 – (Index bits + Offset bits) = 32 – (5 +5) = 22

Tag bits = 2^7 * 22 bits = 2816

Total cache size:

4KB + 2816 bits = 32768 + 2816

35584 bits = 4448 Bytes

Validity bits = 128

Dirty bits = 128

LRU Replacement = 2-bits per line = 256 bits